

VIETNAM - KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY
FACULTY OF COMPUTER SCIENCE



GRADUATION THESIS

**SMART E-COMMERCE WITH AI RECOMMENDATION &
EXPENSE TRACKING**

Full name:	Tran Thi Cam Hoa
Class:	22ITe
Major:	Information Technology
Minor:	Software Engineer
Supervisor:	MS. Ngo Le Quan

Da Nang – 6 June 2026

VIETNAM - KOREA UNIVERSITY OF INFORMATION AND
COMMUNICATION TECHNOLOGY
FACULTY OF COMPUTER SCIENCE

GRADUATION THESIS

SMART E-COMMERCE WITH AI RECOMMENDATION & EXPENSE TRACKING

Full name:	Tran Thi Cam Hoa
Class:	22ITe
Major:	Information Technology
Minor:	Software Engineer
Supervisor:	MSc. Ngo Le Quan

Da Nang – 6 June 2025

SUPERVISOR’S COMMENT

.....

.....

.....

.....

.....

.....

Da Nang, / / 2026

Supervisor

MSc. Ngo Le Quan

ACKNOWLEDGEMENTS

After a period of study and practice at the Vietnam - Korea University of Information and Communication Technology, I would like to express my sincere gratitude and respect to the Board of Directors, the various departments, and the faculty members. Thank you for your enthusiastic guidance, instruction, and for creating favorable conditions for me throughout the process of learning, researching, and completing this graduation thesis.

In particular, I would like to express my deepest gratitude to **MSc. Ngo Le Quan**, who directly guided and supported me during the implementation of this thesis.

I would also like to thank my family and friends for their support and for facilitating my research and surveys, which allowed me to complete this work.

However, due to limitations in my own knowledge and experience, this research inevitably contains shortcomings. I earnestly welcome comments and feedback from the university, my teachers, and peers to help improve this thesis further.

Thank you very much!

TABLE OF CONTENTS

SUPERVISOR’S COMMENT	i
ACKNOWLEDGEMENTS.....	ii
TABLE OF CONTENTS.....	iii
LIST OF TABLES	vi
LIST OF FIGURES	vii
CHAPTER 1: INTRODUCTION.....	1
1.1 Motivation	1
1.1.1 Digital Transformation and Fintech Growth.....	1
1.1.2 Existing Challenges in Digital Finance and E-Commerce.....	1
1.1.3 Fintech Development in Vietnam and Da Nang	2
1.1.4 Proposed Solution: Hmall	3
1.2 Objectives and Contributions.....	3
1.2.2 Specific Objectives	4
1.2.3 Expected Outcomes	6
1.3 Scopes.....	6
1.3.1 Research Object and Target Users	6
1.3.2 Technological Scope.....	7
1.3.3. Boundaries and Deployment Environment	7
1.4. Practical Significance.....	7
1.4.1. Solving Real-World Consumer Problems.....	7
1.4.2. Deployment and Scalability for a Startup	8
CHAPTER 2. LITERATURE REVIEW & BACKGROUND	9
2.1. Industry Overview & Market Demand Survey	9
2.1.1. Digital Transformation Trends in E-Commerce	9
2.1.2. Market Demand Survey and Comparison of Current Solutions.....	9
2.2 Strategic Analysis (PESTEL, SWOT, Porter 5 Forces).....	10
2.2.1 PESTEL Analysis	10
2.2.2. SWOT Analysis	11
2.2.3. Porter's Five Forces Analysis	12
2.3. System Requirements Analysis	13
2.3.1. Functional Requirements	13

2.3.2. Non-Functional Requirements	14
2.4. Theoretical Background	15
2.4.1 Digital Finance & Educational Technology	15
2.4.1 Monorepo Architecture with Docker.....	17
2.5 Application development tools and technology.....	17
2.5.1 User Interface (Front-end).....	18
2.5.2 Server and Database (Back-end)	19
2.5.3 AI Service and Machine Learning.....	21
CHAPTER 3. PRODUCT DESIGN AND DEVELOPMENT	23
3.1 System Architecture.....	23
3.1.1. Service Architecture Overview	23
3.1.2. Component Boundaries and Integrations	23
3.2 System Requirements.....	24
3.2.1 Functional Requirements	24
3.2.1 Non-Functional Requirements	25
3.3 System Design	26
3.3.1 Use Case Model	26
3.3.2 Use case specification	32
3.3.3 Activity Diagram	41
3.3.4 Sequence diagram	52
3.3.5 Entity relationship diagram.....	59
3.3.6 System architecture diagram.....	59
3.3.7 Component architecture diagram	60
3.4 Database Design.....	61
CHAPTER 4. DEPLOYMENT AND BUSINESS MODELS	69
4.1. Deployment Results and User Interface.....	69
4.1.1. Home Page for Guest	69
4.1.2. Login & Register Page	69
4.1.3. General User Screen	70
4.1.4. Shopping Cart Management.....	70
4.1.5. Orders Management.....	70
4.1.6. Wallet & Finance Management	70
4.2. Performance Evaluation & User Feedback	72

4.3. Efficiency Analysis	73
4.4. Startup Strategy and Commercialization	73
4.4.1. Lean Startup Methodology.....	73
4.4.2. Design Thinking Process	74
4.4.3. Business Model Canvas (BMC).....	74
CHAPTER 5. CONCLUSION & PRODUCT ROADMAP	74
5.1. Key Outcomes and Innovations	74
5.1.1. Technical and Functional Milestones.....	75
5.1.2. Innovation and Business Value.....	75
5.3 Recommendations & Product Roadmap.....	76
5.3.1. Phase 1: Technical and Security Optimizations (June 2026 - September 2026).....	76
5.3.2. Phase 2: Live Integrations & AI Upgrades (September 2026 - January 2027)	76
5.3.3. Phase 3: Commercialization and Market Rollout (January 2027 - June 2027)	77
REFERENCES.....	ix

LIST OF TABLES

Table 1 PESTEL Analysis Matrix for Hybrid Fintech E-Commerce Platforms ..	11
Table 2 SWOT Analysis Matrix of the Hmall Platform	12
Table 3 Porter's Five Forces Evaluation for Hmall	13
Table 4 Functional Requirements Specifications Matrix (FRSM).....	14
Table 5: Post Job case specification	32
Table 6: Add to Cart & Checkout Case Specification.....	34
Table 7: Complete Task to Earn Coins Case Specification.....	35
Table 8: Create & Manage Budget Case Specification	36
Table 9: Upgrade Vendor Package Case Specification	37
Table 10: Submit Product Review Case Specification.....	38
Table 11: Chat with Vendor/Support Case Specification.....	39
Table 12: Admin: Moderate Products Case Specification.....	40
Table 13: Admin: Manage Users Case Specification.....	41
Table 14 User Evaluation Metrics Summary (Performance & UX)	73

LIST OF FIGURES

Figure 1 Hybrid Architecture Diagram of the Hmall Platform.....	23
Figure 2: General Use Case	28
Figure 3: End User Use Case.....	29
Figure 4: Vendor Use Case.....	30
Figure 5: Admin Use Case.....	31
Figure 6: Login & Authentication Activity	42
Figure 7: Vendor Post Product Activity	44
Figure 8: User Add to Cart & Checkout Activity.....	46
Figure 9: Complete Task to Earn Coins Activity	49
Figure 10: Submit Product Review Activity	51
Figure 11: User Login & Authentication Sequence Diagram	53
Figure 12: Vendor Post Product Sequence Diagram	54
Figure 13: Add to Cart & Checkout Sequence Diagram	55
Figure 14: Complete Task to Earn Coins Sequence Diagram	56
Figure 15: Submit Product Review Sequence Diagram	57
Figure 16: Upgrade Vendor Package Sequence Diagram	58
Figure 17: Entity relationship diagram	59
Figure 18: System architecture diagram	59
Figure 19: Component architecture diagram	60
Figure 20: Users tale	61
Figure 21: Products table	62
Figure 22: Payments table.....	63
Figure 23: Tasks table.....	64
Figure 24: AI recommendation table	65
Figure 25: Vendor profile table	66
Figure 26 Business Model Canvas for the Hmall Startup.....	74

CHAPTER 1: INTRODUCTION

1.1 Motivation

1.1.1 Digital Transformation and Fintech Growth

The rapid expansion of the global digital economy has fundamentally transformed the way individuals interact with financial services, commercial platforms, and digital technologies. Over the past decade, e-commerce, digital payments, and financial technology (FinTech) solutions have become increasingly integrated into everyday life, creating new opportunities for economic participation and financial inclusion.

1.1.2 Existing Challenges in Digital Finance and E-Commerce

Despite these advancements, existing digital finance and e-commerce ecosystems continue to face several critical challenges that limit their accessibility, effectiveness, and educational value for users, particularly those who are new to digital financial environments.

One of the most significant challenges is the limited level of financial literacy among many users. While digital currencies, online investment platforms, and electronic commerce systems have become more accessible, a substantial portion of the population lacks the practical knowledge and experience required to navigate these technologies safely and effectively. As a result, users are often exposed to unnecessary risks, including poor financial decision-making, fraud, speculative behavior, and losses caused by insufficient understanding of digital financial instruments and market dynamics.

Another major barrier is the complexity associated with entering real-world financial ecosystems. Many financial platforms require users to complete extensive Know Your Customer (KYC) verification procedures, provide personal documentation, maintain minimum capital requirements, and possess a certain degree of technical expertise. These requirements can discourage students, young professionals, and first-time users from exploring digital finance opportunities, thereby limiting broader participation and learning.

Furthermore, the current digital landscape is highly fragmented. Users often rely on multiple independent applications to perform different tasks such as online shopping,

budget management, investment simulation, financial education, and personal productivity. This separation creates a disconnected user experience, reducing efficiency and making it difficult for individuals to develop a comprehensive understanding of how financial and commercial systems interact in practice. The absence of a unified platform also hinders the development of practical skills that require exposure to multiple interconnected financial activities.

In addition, traditional e-commerce and financial platforms frequently lack meaningful personalization capabilities. Although modern systems collect large amounts of user data, many platforms fail to leverage this information effectively to provide tailored recommendations, adaptive learning experiences, or personalized financial guidance. Consequently, users may receive generic content and recommendations that do not align with their individual goals, preferences, or behavioral patterns, reducing engagement and overall user satisfaction.

Perhaps the most critical issue is the inherent risk associated with learning through real financial markets. Individuals seeking to gain experience in digital trading, investment, or online commerce often have no choice but to interact directly with real assets and real money. Without adequate guidance, educational support, or a risk-free environment, beginners may suffer significant financial losses before acquiring the knowledge necessary to make informed decisions. This challenge highlights the need for a secure and educational ecosystem where users can develop practical skills without exposing themselves to actual financial risks.

1.1.3 Fintech Development in Vietnam and Da Nang

These challenges are particularly relevant in Vietnam, a country that is experiencing rapid growth in digital transformation and financial technology innovation. Government initiatives, increasing smartphone adoption, and expanding digital payment infrastructures have accelerated the development of the fintech sector. At the regional level, Da Nang has positioned itself as a leading center for innovation and technology development, with strategic initiatives aimed at fostering a vibrant fintech ecosystem. *Events such as Finance & Technology Week 2025 and VKU DeFi 2025* demonstrate the city's commitment to promoting financial innovation, blockchain technology, and digital entrepreneurship. However, despite these efforts, there remains a noticeable gap between technological advancement and the availability of practical educational

platforms that allow users to safely acquire financial knowledge and digital commerce skills.

1.1.4 Proposed Solution: Hmall

To address these challenges, **Hmall** is proposed as an innovative solution based on the concept of a Hybrid Fintech ecosystem. The platform combines the accessibility and user-friendly experience of modern mobile applications with advanced artificial intelligence technologies to create an integrated environment for digital finance education and e-commerce simulation. Unlike traditional platforms, **Hmall** emphasizes experiential learning, personalization, and risk-free participation through a simulated marketplace where users can explore financial concepts, practice decision-making, and engage in commercial activities without the fear of real financial loss.

By integrating e-commerce functionalities, personal financial management tools, educational resources, and AI-powered recommendation systems within a single platform, **Hmall** seeks to bridge the gap between theoretical financial knowledge and practical application. The platform aims to empower users with the skills and confidence necessary to participate in the digital economy while promoting responsible financial behavior, continuous learning, and long-term economic inclusion. Through this approach, **Hmall** contributes not only to individual user development but also to the broader objective of supporting Vietnam's emerging fintech ecosystem and Da Nang's vision of becoming a leading regional hub for financial technology innovation.

1.2 Objectives and Contributions

1.2.1 General Objective

The primary aim of this thesis is to design, develop, and evaluate **Hmall**, a Hybrid Fintech Platform that integrates simulated e-commerce and personal finance management within a unified digital ecosystem. The platform is intended to provide users with a safe and accessible environment in which they can explore digital commerce, financial planning, and virtual asset management without being exposed to the risks associated with real financial markets.

Hmall seeks to democratize financial literacy by eliminating common barriers to participation, such as capital requirements, complex onboarding procedures, and limited practical learning opportunities. By leveraging modern mobile technologies, cloud-

based infrastructure, and artificial intelligence-driven personalization, the platform aims to deliver an engaging and educational user experience that supports both learning and skill development. Furthermore, the system is designed to demonstrate how a Hybrid Fintech model can effectively combine financial education, e-commerce simulation, and intelligent recommendation services to foster responsible financial behavior and improve digital financial competence among users.

1.2.2 Specific Objectives

1.2.2.1 Architectural Design

The first objective of this research is to design and implement a scalable, maintainable, and extensible system architecture capable of supporting multiple interconnected fintech and e-commerce services. To achieve this, the project adopts a Hybrid Monorepo architecture that enables efficient development, code sharing, and streamlined deployment processes while maintaining clear separation between business domains.

The architecture incorporates PostgreSQL, managed through Supabase, as the primary data storage solution to ensure reliability, consistency, and efficient data management. Additionally, the system integrates dedicated service layers for authentication, financial transactions, virtual asset management, and e-commerce operations. An independent Artificial Intelligence service is incorporated to handle recommendation generation and predictive analytics, while modular microservices are employed to support scalability and future expansion. This architectural design aims to provide a robust technological foundation capable of accommodating increasing numbers of users, services, and data interactions.

1.2.2.2 AI Recommendation Implementation

The second objective is to develop and integrate an intelligent recommendation system that enhances user engagement and personalization throughout the platform. Traditional financial and e-commerce systems often provide generic content that fails to address the unique preferences and behaviors of individual users. To overcome this limitation, **Hmall** incorporates a hybrid recommendation framework that combines multiple recommendation techniques.

Specifically, the system utilizes collaborative filtering to identify patterns among users with similar behaviors, content-based filtering to analyze product and user

attributes, and rule-based recommendation mechanisms to enforce business logic and educational objectives. By integrating these approaches, the platform can generate personalized product suggestions, virtual investment opportunities, budgeting recommendations, and educational content tailored to each user's interests and activities. The implementation of AI-powered recommendations is expected to improve user satisfaction, increase platform engagement, and provide a more adaptive learning experience.

1.2.2.3. User Experience Optimization

The third objective focuses on designing an intuitive, accessible, and user-centered experience that encourages participation from users with varying levels of financial and technological knowledge. Since one of the primary goals of **Hmall** is to reduce barriers to entry, particular attention is given to user interface (UI) and user experience (UX) design principles.

The platform adopts a mobile-first development strategy to accommodate the growing reliance on smartphones as the primary means of accessing digital services. Key features include simplified navigation, responsive interfaces, real-time feedback mechanisms, and seamless transitions between different functional modules. In addition, the system incorporates virtual currency management capabilities that allow users to practice financial decision-making in a risk-free environment. By ensuring smooth integration between e-commerce, finance, and recommendation services, the platform aims to provide a cohesive and engaging experience that promotes continuous learning and exploration.

1.2.2.4. Prototype/MVP Development

The fourth objective is to develop a Minimum Viable Product (MVP) of **Hmall** that integrates the core functionalities of simulated e-commerce, personal finance management, virtual asset operations, and AI-powered recommendations. The MVP serves as a proof-of-concept implementation that demonstrates the feasibility of the proposed Hybrid Fintech model and provides a foundation for future enhancements and commercialization.

1.2.2.5. System Validation and Evaluation

The final objective is to evaluate the effectiveness, reliability, and feasibility of the proposed Hybrid Fintech model through comprehensive system testing and validation.

This process involves assessing the platform's functionality, security, performance, scalability, and overall user experience under realistic operating conditions.

A series of validation activities will be conducted, including functional testing to verify system requirements, security testing to ensure data protection and user privacy, and performance testing to evaluate responsiveness and resource utilization. Additionally, the effectiveness of the recommendation engine and the overall usability of the platform will be assessed to determine whether the system successfully meets its educational and operational goals. Through these evaluations, the research aims to demonstrate the practicality of **Hmall** as a modern fintech education platform and validate the potential of Hybrid Fintech architectures for future digital financial learning environments.

1.2.3 Expected Outcomes

By achieving the objectives outlined above, this thesis is expected to deliver a fully functional Hybrid Fintech platform that integrates simulated e-commerce, personal finance management, and AI-driven personalization within a single ecosystem. The project will provide evidence that a risk-free, intelligent, and user-centered environment can effectively support financial literacy development while maintaining the usability, scalability, and performance standards required by contemporary digital applications.

Furthermore, the proposed platform is designed with future commercialization potential in mind. Its modular architecture and AI-driven services provide a foundation for expansion into a sustainable fintech education and digital commerce ecosystem that can support partnerships with educational institutions and industry stakeholders.

1.3 Scopes

1.3.1 Research Object and Target Users

- **Research Object:** Hybrid software architecture, Collaborative Filtering algorithms using Cosine similarity matrices, TF-IDF keyword extraction, and budget tracking and alerts.
- **Target Users:**
 - **Buyers/Customers:** Young consumers, students, and office workers who shop online frequently and need automated tools to manage their daily budgets.

- Vendors/Shop Owners: Small-scale businesses and individual sellers who want to reach customers through the platform's AI recommendations with low operational costs.
- Administrators: System operators who moderate products, manage user roles, and monitor transaction ledgers.

1.3.2 Technological Scope

The project is built on a modern, unified technology stack:

- Mobile Client: Cross-platform application built with React Native using the Expo SDK and TypeScript to optimize development speed and native interface rendering.
- Backend Services:
 - Core RESTful APIs developed in Node.js (Express)
 - Media assets (product images) managed via Cloudinary API.
- Database & Cloud Infrastructure: Powered by Supabase (PostgreSQL) to maintain financial ledger consistency and support real-time data sync.
- AI Service Engine: Run as an independent TypeScript Node.js service, using the Natural NLP library and the ml-matrix linear algebra library to train and execute models.

1.3.3. Boundaries and Deployment Environment

- Boundaries: The virtual coin wallet is a simulated virtual currency system. Payment integrations (VNPay/MoMo) run in sandbox/mock modes to demonstrate financial workflows without executing real money transfers. The AI model is trained on simulated interaction histories gathered from pilot user trials.
- Deployment: Backend services and the AI engine are containerized using Docker and run locally. The mobile application is run on physical test devices using the Expo Go client.

1.4. Practical Significance

1.4.1. Solving Real-World Consumer Problems

Hmall offers a practical solution to a major challenge of personal financial management: manual data entry. By integrating the expense tracker directly into the e-commerce client, transaction categories are parsed and updated in the user's dashboard

immediately upon checkout. This helps users monitor their cash flow effortlessly, encouraging healthier spending habits.

1.4.2. Deployment and Scalability for a Startup

The hybrid architecture utilizes open-source technologies and low-cost cloud services (Supabase, Cloudinary), minimizing initial infrastructure costs for a startup. Furthermore, its modular design enables high scalability. When user traffic increases, the AI recommender or order wallet engines can be scaled out independently as microservices without refactoring the frontend or database schemas.

CHAPTER 2. LITERATURE REVIEW & BACKGROUND

2.1. Industry Overview & Market Demand Survey

2.1.1. *Digital Transformation Trends in E-Commerce*

The growth of the global digital economy has driven e-commerce models from simple transactional portals toward personalized and integrated utility ecosystems. Digital transformation in e-commerce is no longer just about listing products online; it represents a convergence of several breakthrough technologies:

- **Cloud Computing:** Provides flexible infrastructure and distributed databases capable of autoscaling to handle high transactional traffic without server bottlenecks.
- **Artificial Intelligence (AI):** Serves as the analytical core to parse user behavior, classify large datasets, and deliver real-time recommendations.
- **Fintech & Digital Wallets:** Integrating frictionless digital payment gateways directly into checkout flows increases conversion rates and reduces order cancellation rates.

2.1.2. *Market Demand Survey and Comparison of Current Solutions*

To define the unique value proposition of **Hmall**, the research team analyzed and compared current e-commerce and personal finance applications.

Standard E-Commerce Platforms (standard, Lazada, Tiki)

- **Advantages:** Vast product catalogs, established merchant bases, and mature logistics systems. Gamified point systems (like standard Coins) effectively drive repeat purchases.
- **Disadvantages:**
 - Lack of personal finance management tools. These platforms encourage constant spending using discounts and flash sales, which can lead to impulsive shopping.
 - Closed reward loops. Users cannot track ledger entries or analyze their spending habits through visual charts.

Standalone Personal Finance Apps (Money Lover, Spendee, MISA MoneyKeeper)

- **Advantages:** Visual budget charting, multi-category limits, and clean financial reports.

- Disadvantages:
 - Rely entirely on manual entry for every transaction. This creates a high behavior barrier, causing users to abandon the app after a few weeks.
 - No direct e-commerce integration. The apps cannot sync digital receipts automatically from external shopping platforms.

Integrated Hmall Solution

Hmall acts as a bridge between these two domains:

1. Automated Bookkeeping: When a user purchases a product on **Hmall**, the order and cost details are automatically categorized under "Shopping" inside the Expense Tracker without manual input.
2. Transparent Loyalty Points: A 1:1 reward coin system (1 coin = 1 VND) with an immutable transaction ledger, allowing direct discount offsets on cash checkout, helping users save money while tracking expenses.

2.2 Strategic Analysis (PESTEL, SWOT, Porter 5 Forces)

To assess the viability and competitiveness of the **Hmall** platform as a startup project, we conducted strategic business evaluations.

2.2.1 PESTEL Analysis

The PESTEL model evaluates the macro-environmental factors affecting the growth of the **Hmall** platform in the digital commerce space.

Factor	Context & Trends	Impact on Hmall
Political	Government initiatives promoting cashless payments and national digital transformations.	Creates a favorable regulatory framework for integrated Fintech startups.
Economic	Rising disposable income in developing markets driving e-commerce and wealth management needs.	Consumers are willing to spend online but require smarter budget tracking tools.
Social	Tech-savvy Gen Z and Millennial majorities who	Young consumers are prone to impulsive spending but open to gamified AI finance tools.

	value speed and convenience.	
Technological	Wide 4G/5G coverage, high smartphone adoption, and mature open-source AI libraries.	Lowers initial infrastructure costs; models can run efficiently on lightweight cloud engines.
Environmental	High demand for paperless billing and eco-friendly consumer habits.	Digitizing 100% of receipts and finance records eliminates paper waste.
Legal	Stringent personal data protection and digital transaction security regulations.	Requires secure authentication and data isolation protocols (such as Supabase RLS).

Table 1 PESTEL Analysis Matrix for Hybrid Fintech E-Commerce Platforms

2.2.2. SWOT Analysis

The SWOT model identifies the internal strengths and weaknesses of **Hmall**, alongside external opportunities and threats.

Strategic Factors	Details	Strategic Action
Strengths	1. Unique 2-in-1 integration of shopping and finance. 2. Service-oriented modular backend optimizing performance. 3. Transparent 1:1 virtual coin discount. 4. AI-driven personalization and budget tracking and alerts.	Emphasize the unique integrated value proposition in marketing campaigns; leverage AI as a primary user acquisition tool.
Weakness	1. Virtual wallet currently operates in a simulated sandbox. 2. Low brand awareness as a new market entry. 3. Limited initial user history (cold start problem).	Focus on refining the core UX; implement a hybrid recommendation model to mitigate the cold start issue.

Opportunities	1. E-commerce market growing consistently at over 20% annually. 2. Increasing public awareness of personal finance. 3. Regulatory sandbox policies supporting Fintech innovation.	Partner with small boutique vendors to expand the marketplace; introduce gamified tasks to increase user engagement.
Threats	1. Strong competition from large e-commerce platforms with capital reserves. 2. Cybersecurity risks and potential balance manipulation attacks. 3. Initial user skepticism toward new digital financial systems.	Enforce append-only transaction ledger records; implement secure JWT tokens and encrypted API communication.

Table 2 SWOT Analysis Matrix of the **Hmall** Platform

2.2.3. Porter's Five Forces Analysis

This model assesses the competitive landscape of the market segment **Hmall** is entering.

Competitive Force	Threat Level	Market Reality for Hmall
1. Rivalry among existing competitors	High	Major platforms like standard and Lazada hold dominant market shares. However, their lack of integrated budget tracking tools leaves a niche market open for Hmall .
2. Threat of new entrants	Medium	Building a basic e-commerce mobile application is straightforward, but developing secure transaction engines and accurate AI recommenders requires specialized expertise.

3. Bargaining power of buyers	High	E-commerce users have low switching costs and follow discounts. Hmall mitigates this with a transparent coin economy and automated expense insights to drive retention.
4. Bargaining power of suppliers	Low-Medium	Small merchants seek channels with lower commission fees. Hmall attracts merchants by charging a flat 10% commission and returning 90% in equivalent coins.
5. Threat of substitute products	Medium	Users can continue using separate shopping and budgeting apps. However, the friction of manual data entry makes Hmall 's automated sync highly attractive.

Table 3 Porter's Five Forces Evaluation for **Hmall**

2.3. System Requirements Analysis

2.3.1. Functional Requirements

The system must satisfy all core modules including E-commerce, wallet ledgers, AI recommendations, and budget forecasting.

Req ID	Module	Feature	Business Specification
FR-01	User Accounts	Signup & Role Management	User registration and JWT-based authentication. Toggles between User, Vendor, and Admin roles. Grants 10,000 welcome coins upon registration.
FR-02	E-Commerce	Product Inventory	Vendors can upload products (title, description, price in VND, inventory stock, images via Cloudinary). Admins approve products for sale.
FR-03	E-Commerce	Cart & Checkout	Users add items to a dynamic shopping cart, manage quantities, and select items for order placement.

			Checkout screen displays cost breakdowns.
FR-04	Fintech & Wallet	Coin Discount Offset	Users can toggle "Use Coins" at checkout. The system calculates coin deductions at a 1:1 rate to offset the cash VND total.
FR-05	Fintech & Wallet	Ledger Audit Trail	Append-only logging of coin transactions. Automatically credits 90% of gross order values in coins to vendors on payment confirmation.
FR-06	Personal Finance	Auto Expense Tracker	Automatically creates expense logs from e-commerce purchases. Allows users to set monthly category budgets (e.g., Food, Shopping).
FR-07	AI Engines	Product Recommender	Displays personalized recommendations on the homepage using a hybrid collaborative/content-based microservice.
FR-08	AI Engines	Spending Forecasting	Analyzes monthly cash flow, flags unusual transactions, and sends notifications when the budget limit is approached.

Table 4 Functional Requirements Specifications Matrix (FRSM)

2.3.2. Non-Functional Requirements

- Security:
 - All protected API endpoints require JWT authentication sent via the Authorization: Bearer <token> header.
 - Database-level separation of client records must be enforced via Supabase Row Level Security (RLS) policies.
- Consistency & Data Integrity:

- Financial transactions and stock updates must run inside ACID database transactions (PostgreSQL RPC) to prevent data mismatch.
- Performance:
 - Average response latency for catalog browsing and cart updates must remain under 200ms.
 - AI recommender response times must remain under 1 second to ensure a smooth homepage load.
- Compatibility: The client app must run on Android (version 8.0+) and iOS (version 13+) via Expo SDK.

2.4. Theoretical Background

2.4.1 Digital Finance & Educational Technology

Digital finance and educational technology are two rapidly evolving domains that increasingly intersect in modern digital ecosystems. The integration of these domains enables the development of platforms that not only facilitate financial transactions but also support experiential learning and behavioral development.

Hmall is designed at the intersection of these two fields, combining financial simulation, e-commerce interaction, and intelligent personalization to create a comprehensive learning environment.

2.4.1.1 Modern Mobile Application Development

Mobile technology has become the primary gateway to digital services, particularly in emerging economies such as Vietnam, where smartphone penetration is exceptionally high. As a result, mobile-first design has become a fundamental requirement for modern digital platforms.

React Native provides a cross-platform development framework that enables developers to build applications for both Android and iOS using a unified codebase. This significantly improves development efficiency while maintaining near-native performance. Expo further enhances this ecosystem by providing tools for rapid development, testing, deployment, and over-the-air updates.

The component-based architecture of React promotes modular design and code reuse, allowing complex interfaces to be built in a structured and maintainable manner. In addition, state management tools such as TanStack Query enable efficient handling of server-side data through caching, synchronization, and background updates.

For **Hmall**, these technologies ensure high accessibility, rapid development cycles, and consistent user experiences across devices, which are essential for educational and financial simulation applications.

2.4.1.2 AI-Powered Personalization and Recommendation Systems

Artificial intelligence plays a crucial role in enhancing user engagement and personalization within digital platforms. In the context of **Hmall**, AI is used to deliver adaptive recommendations, financial insights, and personalized learning experiences.

The system employs a hybrid recommendation approach that combines content-based filtering, collaborative filtering, and rule-based systems. Content-based filtering analyzes item attributes and user preferences, while collaborative filtering identifies patterns among users with similar behaviors. Rule-based logic is used to enforce financial constraints and educational objectives.

This hybrid approach enables the system to generate highly relevant recommendations, including product suggestions, budget planning advice, transaction categorization, and gamified learning tasks. As a result, users receive a more personalized and adaptive experience that enhances both engagement and educational effectiveness.

2.4.1.3 Supabase (Backend-as-a-Service)

Supabase is an open-source Backend-as-a-Service platform built on PostgreSQL. It provides a complete backend infrastructure, including database management, authentication, storage, and real-time capabilities.

The use of PostgreSQL ensures strong relational data modeling, making it suitable for complex financial and e-commerce systems. Supabase authentication supports secure user management through email, OAuth, and token-based authentication mechanisms.

Real-time subscriptions enable dynamic updates for features such as notifications and transactional updates. Additionally, Supabase provides TypeScript integration, ensuring type safety and consistency across the system.

In **Hmall**, Supabase serves as the central data layer, ensuring a single source of truth for all user, product, and financial data.

2.4.1 Monorepo Architecture with Docker

Hmall adopts a monorepo architecture in which all system components are maintained within a single repository. This approach improves code consistency, simplifies dependency management, and enables better collaboration across development teams.

Each service within the system is modularized, allowing independent development and deployment of frontend, backend, and AI components. This modular structure enhances scalability and maintainability.

Docker is used to containerize all services, ensuring consistent runtime environments across development, testing, and production stages. Docker Compose further simplifies orchestration by allowing all services to be launched through a single configuration.

This combination of monorepo structure and containerization ensures high system reliability, reproducibility, and deployment efficiency.

2.5 Application development tools and technology

This section presents a detailed overview of the software development tools, frameworks, libraries, and infrastructure components used in the implementation of the **Hmall** platform. The selected technology stack is designed to ensure high system performance, scalability, maintainability, and type safety while enabling seamless integration across multiple system layers, including frontend, backend, and artificial intelligence services.

Given the hybrid nature of **Hmall**, which combines e-commerce simulation, financial management, and AI-driven personalization, the technology stack must

support both rapid development and long-term extensibility. Therefore, a modern full-stack JavaScript/TypeScript ecosystem, combined with cloud-native and containerized infrastructure, is adopted as the foundation of the system.

2.5.1 User Interface (Front-end)

The frontend layer of **Hmall** is developed using a mobile-first architecture, reflecting the growing dominance of mobile devices as the primary access point for digital services, particularly in emerging markets such as Vietnam. The objective of the frontend design is to provide a responsive, intuitive, and highly interactive user experience that supports both educational and transactional activities.

React Native & Expo Ecosystem

The core framework used for frontend development is **React Native 0.81.5**, which enables the development of cross-platform mobile applications using a single codebase. This approach significantly reduces development effort while maintaining near-native performance on both iOS and Android platforms.

To complement React Native, **Expo 54** is integrated as the development and deployment toolchain. Expo provides a comprehensive set of features including hot reloading, over-the-air updates, push notification services, and simplified access to native device APIs. This accelerates the development lifecycle and improves iteration speed, which is particularly important for a rapidly evolving educational platform such as **Hmall**.

The entire frontend codebase is implemented using **TypeScript**, a statically typed superset of JavaScript. TypeScript enhances code reliability by introducing compile-time type checking, reducing runtime errors, and improving long-term maintainability in large-scale applications.

Navigation and State Management

To manage navigation flow within the mobile application, **React Navigation 6** is used. It provides a flexible system for implementing stack-based navigation, tab-based navigation, and deep linking, ensuring smooth transitions between different application modules such as marketplace, finance dashboard, and AI recommendation views.

For server-state management, **TanStack Query 5** is employed. This library plays a critical role in handling data fetching, caching, synchronization, and background updates. By abstracting complex asynchronous operations, it significantly improves application responsiveness and reduces redundant network requests.

In addition, **AsyncStorage** is used to persist lightweight local data, such as user preferences, authentication tokens, and temporary offline state. This ensures continuity of user experience even in unstable network conditions.

UI Components and Styling System

The user interface design follows a custom **HSL-based design system**, which ensures visual consistency, accessibility, and scalability across the entire application. This design approach allows dynamic theme control and improves usability across different device types and lighting conditions.

To enhance user interaction, several supporting libraries are integrated:

- **React Native Gesture Handler** enables smooth and native-like gesture interactions, improving usability in touch-based environments.
- **Expo Image Picker** allows users to upload and manage images, which is essential for product listings and user profiles within the simulated marketplace.
- **React Native Chart Kit** is used to visualize financial data, including spending patterns, budget distribution, and performance analytics, thereby enhancing the educational value of the platform.

Overall, the frontend architecture prioritizes usability, performance, and accessibility, ensuring that users with varying levels of technical expertise can effectively engage with the system.

2.5.2 Server and Database (Back-end)

The backend architecture of **Hmall** is designed using a dual-layer approach that combines lightweight API services with enterprise-grade modular frameworks. This hybrid backend structure ensures both development flexibility and long-term scalability.

Express.js Backend (Core API Layer)

The primary backend service is built using **Node.js 18+ with Express 4.18**, a minimalist and high-performance web framework suitable for building RESTful APIs. This layer is responsible for handling core business logic, including authentication, transaction processing, product management, and communication with external services.

Database interactions are managed through the **Supabase JavaScript Client (v2.38)**, which provides a type-safe interface to the PostgreSQL database. Authentication is implemented using **JWT (JSON Web Tokens)** to ensure secure and stateless session management.

Security and reliability are further enhanced through the integration of:

- **bcryptjs** for secure password hashing
- **Express Rate Limit** for mitigating brute-force and abuse attacks
- **Swagger UI Express** for automatic API documentation generation at runtime

For external integrations, the system incorporates:

- **Cloudinary SDK** for image storage and optimization
- **Firebase Admin SDK** for push notification delivery
-

This layer is optimized for speed, simplicity, and rapid development cycles.

Deleted Section

In addition to Express.js, **Hmall** also incorporates Node.js, a progressive Node.js framework designed for building scalable and maintainable server-side applications. The architecture inspired by Angular, making it suitable for enterprise-level system expansion.

This dual-backend strategy allows **Hmall** to balance rapid prototyping (Express.js) with structured long-term scalability (Express).

Database and Storage Layer

The core database system is built on **Supabase PostgreSQL**, which provides a fully managed relational database with strong ACID compliance. This ensures data consistency across critical modules such as transactions, user accounts, and marketplace operations.

For media storage, **Cloudinary** is used to handle image uploads, transformations, and content delivery optimization.

Additionally, **Redis (planned integration)** is considered for caching frequently accessed data, which will further improve system performance and reduce database load in future scaling phases.

2.5.3 AI Service and Machine Learning

The AI subsystem of **Hmall** is implemented as an independent microservice to ensure modularity, scalability, and separation of concerns. This service is responsible for generating personalized recommendations, analyzing user behavior, and categorizing financial transactions.

Core Technologies

The AI service is developed using **Node.js with Express**, providing a lightweight API layer for model inference and data processing. Several specialized libraries are integrated to support machine learning and natural language processing tasks:

- **ml-matrix** is used for matrix computations in collaborative filtering algorithms
- **natural** provides NLP capabilities for keyword extraction and text analysis
- **simple-statistics** supports statistical scoring and behavioral analysis
- **node-cron** enables scheduled tasks such as periodic model updates and retraining

Recommendation System Design

The AI system is based on a **hybrid recommendation architecture**, which combines multiple techniques to improve accuracy and personalization:

- 1. Content-Based Filtering**

This approach analyzes product attributes such as category, price, and description to recommend items similar to those previously interacted with by the user.

2. **Collaborative Filtering**

This method identifies patterns among users with similar behavioral profiles to generate recommendations based on collective preferences.

3. **Rule-Based Recommendations**

This layer enforces business logic such as budget constraints, educational objectives, and diversification strategies.

4. **Transaction Categorization**

Natural language processing techniques are applied to classify financial transactions based on textual descriptions, improving financial tracking and analytics.

This combination enables **Hmall** to deliver highly personalized and context-aware recommendations across both e-commerce and financial simulation modules.

CHAPTER 3. PRODUCT DESIGN AND DEVELOPMENT

3.1 System Architecture

3.1.1. Service Architecture Overview

The **Hmall** system is designed on a Service-Oriented Architecture (SOA), separating core components into distinct, independent services. This boundary increases resilience, isolates data security scopes, and enables modular deployments.

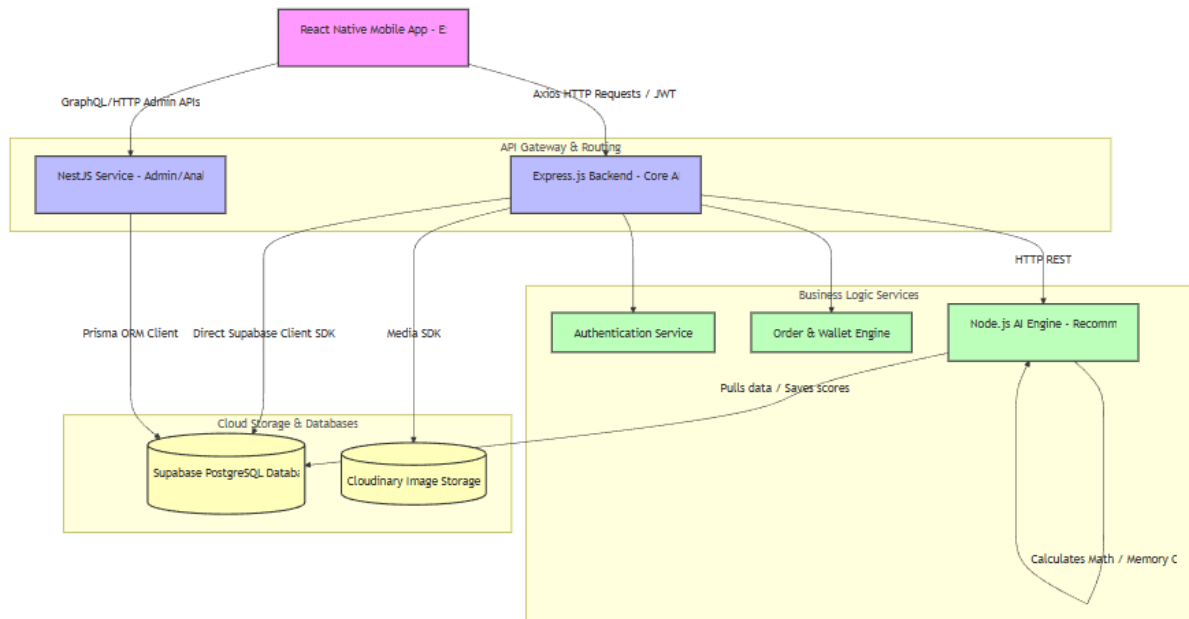


Figure 1 Hybrid Architecture Diagram of the **Hmall** Platform

3.1.2. Component Boundaries and Integrations

1. **Mobile Client (React Native & Expo Client):** Renders the user interface and captures buyer actions. The client maps inputs to REST endpoints, calling the API gateways via Axios clients utilizing customized Interceptors to automatically attach JWT authorization headers.

2. **API Gateway Layer (Express.js & Express):**

- **Express.js Core Service:** Processes high-frequency operations such as logins, catalog searches, shopping cart updates, order creation, and personal finance entries.

- Express Administration Service: Processes administrative actions, vendor analytics, and back-office reports, utilizing Node.js's dependency injection and clean architecture.

3. AI Node.js Service Microservice: Executes as a separate Node.js service. It receives execution triggers from the Express gateway. The service handles training product recommendation matrices, categorizing raw expenses, and running regression projections.

4. Database & Storage Layers (Supabase & Cloudinary):

- Supabase PostgreSQL Database: Stores core relational data including user profiles, products, order states, wallet balances, and financial transaction records.
- Cloudinary Store: Manages image assets, transforming and serving vendor uploads to the mobile client via CDN URLs.

3.2 System Requirements

This section defines the functional and non-functional requirements of the **Hmall** system, derived from the hybrid nature of the platform.

3.2.1 Functional Requirements

A. End User Requirements

The system must support the following capabilities for End Users:

- Secure authentication and profile management
- Virtual wallet operations including balance tracking and transaction history
- E-commerce functionalities such as browsing, searching, filtering, cart management, and order tracking
- Task participation and reward collection mechanisms
- Budget creation, monitoring, and visualization
- AI-driven recommendations and financial insights
- Real-time communication and notification systems

- Product rating and review submission

B. Vendor Requirements

The system must enable vendors to:

- Register and upgrade to vendor accounts
- Manage shop profiles and vendor packages
- Perform full product lifecycle management
- Handle order processing and customer communication
- Access analytics and business performance dashboards

C. Admin Requirements

The system must provide administrative capabilities including:

- User monitoring and moderation
- Product approval and rejection workflows
- Task creation and reward configuration
- System-wide analytics and reporting
- Vendor management and platform governance

3.2.1 Non-Functional Requirements

Performance & Scalability

The system is designed to support high concurrency and low latency operations. Core interactions such as browsing, searching, and messaging must maintain response times under 500 milliseconds for the majority of requests. The architecture must support at least 1000 concurrent users without significant degradation in performance.

The modular structure allows independent scaling of frontend, backend, and AI services.

Security & Data Integrity

Security is a critical requirement of the system due to its financial simulation nature. The system must implement:

- Secure password hashing using bcrypt with high salt rounds
- JWT-based authentication with token expiration policies
- Supabase Row Level Security (RLS) for data access control
- Rate limiting on sensitive endpoints to prevent abuse
- Verification mechanisms for payment gateway integrations

Usability and Accessibility

The system must be optimized for mobile-first usage, ensuring intuitive navigation and responsive design. Educational elements should be subtly integrated to enhance financial literacy. Gamification mechanisms are used to improve engagement and user retention.

Reliability and Availability

The system must ensure high availability with a target uptime of 99% for core services. Robust error handling mechanisms must be implemented to provide meaningful feedback to users. Automated backup systems ensure data safety and recovery capabilities.

3.3 System Design

This section describes the structural and behavioral design of the **Hmall** system through modeling techniques such as Use Case analysis.

3.3.1 Use Case Model

The system is designed around three primary actors: End User, Vendor, and Admin. Each actor interacts with the system through a defined set of use cases.

End User Use Cases

End Users interact with the system through the following functionalities:

- Authentication and profile management
- Wallet operations and virtual currency management

- Product browsing, searching, and purchasing
- Task participation and reward collection
- Budget creation and financial tracking
- AI-powered recommendations and insights
- Real-time messaging and notifications
- Product reviews and ratings

Vendor Use Cases

Vendors are responsible for:

- Account upgrade and authentication
- Shop configuration and management
- Product lifecycle management
- Order processing and customer communication
- Sales analytics and performance monitoring

Admin Use Cases

Administrators handle:

- User management and moderation
- Product approval workflows
- Task system configuration
- Platform analytics and monitoring

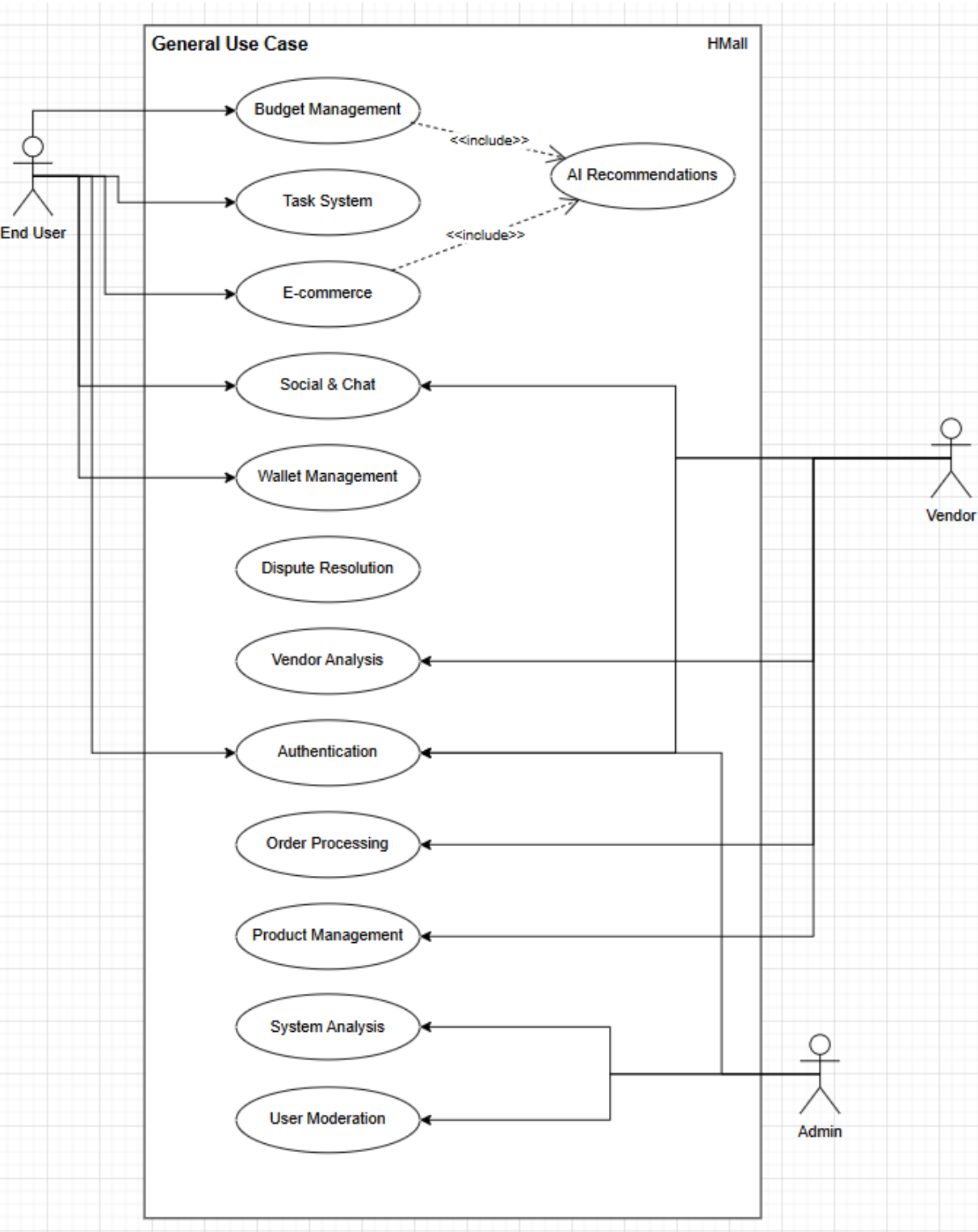


Figure 2: General Use Case

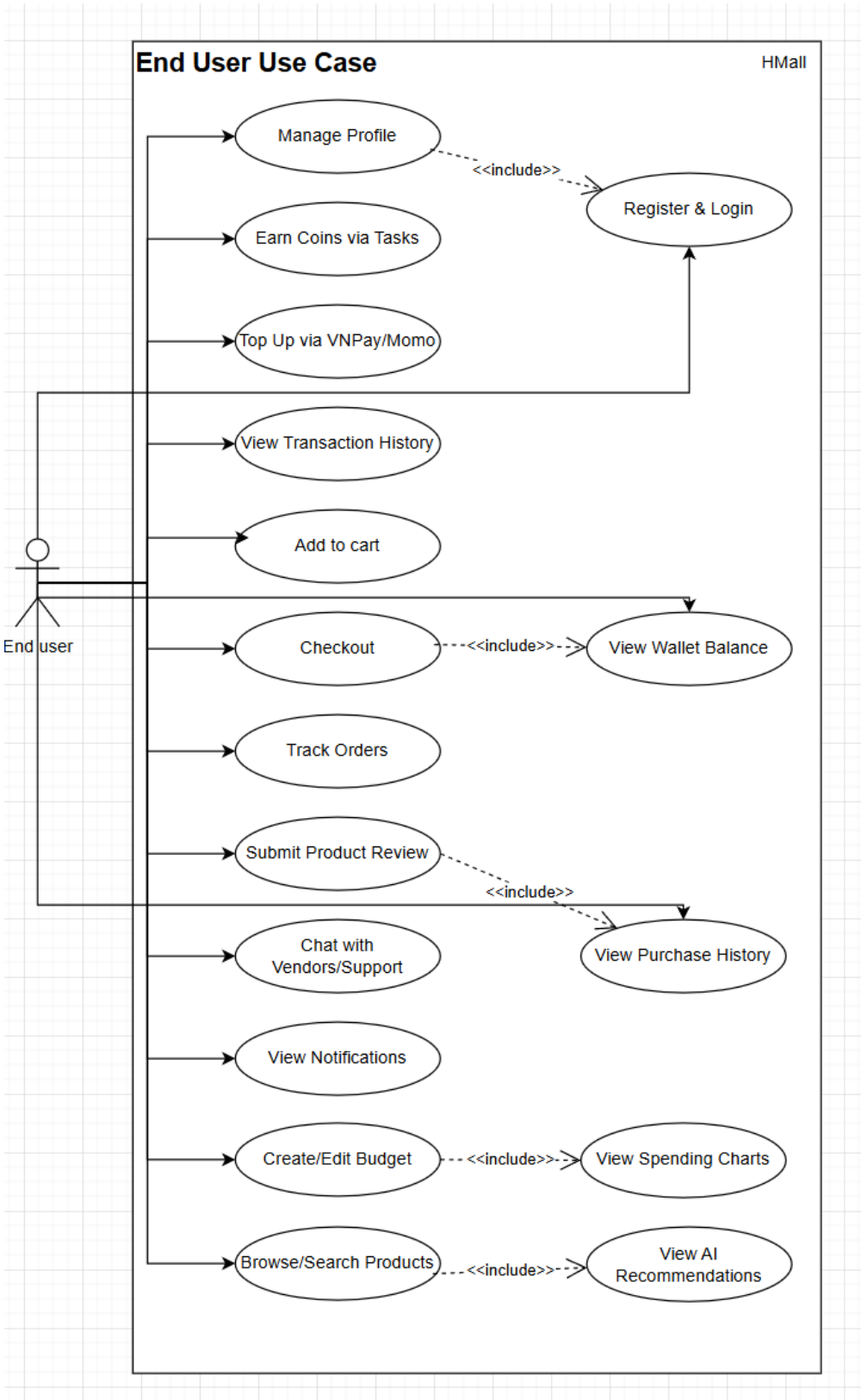


Figure 3: End User Use Case

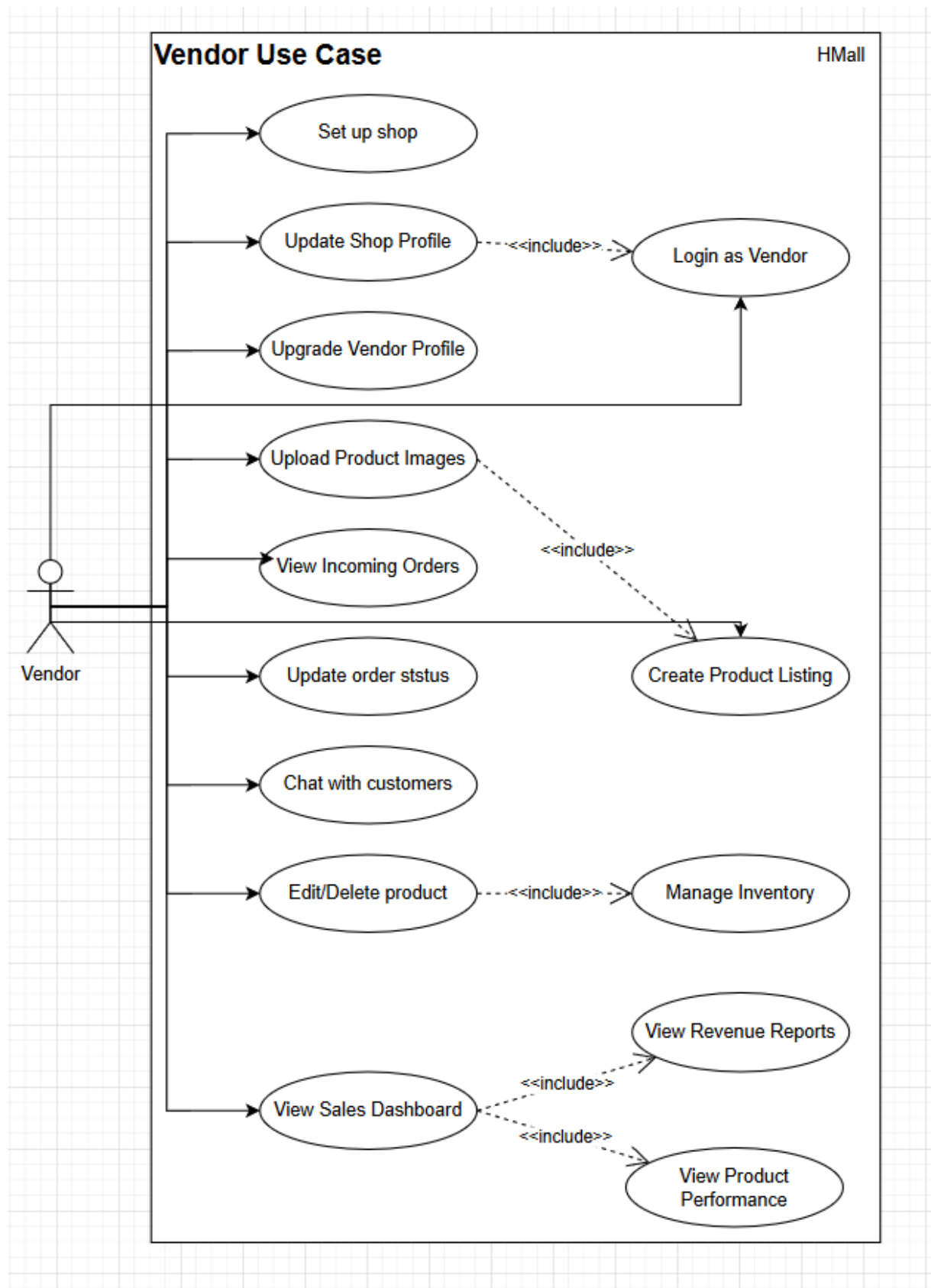


Figure 4: Vendor Use Case

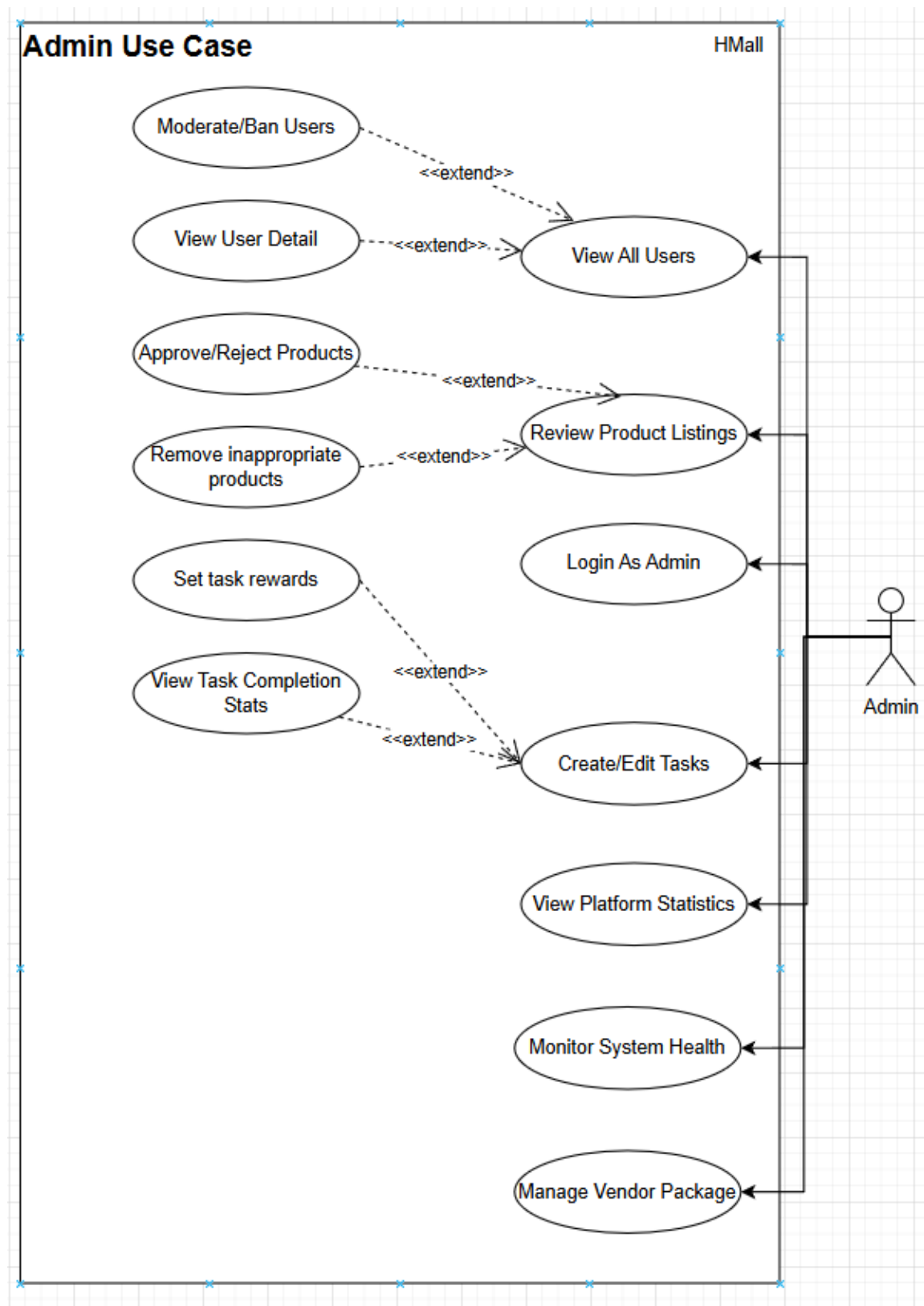


Figure 5: Admin Use Case

3.3.2 Use case specification

Post product case specification

Use case	Post Product
Description	- As a Vendor, I want to post a product to sell on Hmall marketplace
Actor	- Vendor
Pre-Condition	- Vendor must have an account and logged in to the system - Vendor should have an active shop
Result	- A new product was added to the Database - Product is visible in the marketplace
Scenario	--- Vendor actions --- 1. Vendor login to the system 2. Navigate to Vendor Dashboard 3. Click on Add New Product button 4. Enter all necessary information (name, description, price, category, stock) 5. Upload product images 6. Preview product data 7. Click Publish Product button --- System actions --- 1. Checks the validation data entered by the vendor 2. Uploads images to Cloudinary 3. Saves new product to Database 4. Makes product visible in the marketplace

Table 5: Post Job case specification

Add to Cart & Checkout Case Specification

Use case	Add to Cart & Checkout
Description	- As a User, I want to add products to cart and checkout using virtual currency
Actor	- End User
Pre-Condition	- User must have an account and logged in to the system - User must have sufficient balance in wallet
Result	- Products are ordered - Wallet balance is deducted - Order is created with PENDING status
Scenario	--- User actions --- 1. User login to the system 2. Browse and search products in the marketplace 3. Click Add to Cart button on desired products 4. Navigate to Shopping Cart page 5. Review cart items and total amount 6. Click Checkout button 7. Choose payment method (Wallet Payment or Direct Transfer) 8. Confirm the order --- System actions --- 1. Checks if user has sufficient balance 2. Validates product stock availability 3. Creates a new Order with PENDING status 4. Deducts amount from user's wallet

	5. Saves transaction history to Database 6. Sends notification to vendor 7. Sends order confirmation to user
--	--

Table 6: Add to Cart & Checkout Case Specification

Complete Task to Earn Coins Case Specification

Use case	Complete Task to Earn Coins
Description	- As a User, I want to complete tasks to earn virtual coins
Actor	- End User
Pre-Condition	- User must have an account and logged in to the system
Result	- Task is marked as completed - User receives reward coins to wallet
Scenario	--- User actions --- 1. User login to the system 2. Navigate to Tasks page 3. Browse available tasks (Verification Tasks or Action Tasks) 4. Select a task to complete 5. Follow task instructions and complete requirements 6. Submit task completion

	--- System actions --- 1. Validates task completion according to validation rules 2. If task is approved: - Creates transaction record with EARN type - Adds reward coins to user's wallet - Updates task status to COMPLETED 3. If task is rejected: - Updates task status to REJECTED - Notifies user to try again
--	--

Table 7: Complete Task to Earn Coins Case Specification

Create & Manage Budget Case Specification

Use case	Create & Manage Budget
Description	- As a User, I want to create and manage budgets to track spending
Actor	- User
Pre-Condition	- User must have an account and logged in to the system
Result	- Budget is created and saved - Spending is tracked against budget limit

Scenario	---User actions --- 1. User login to the system 2. Navigate to Budget Management page 3. Click Create New Budget button 4. Enter budget details (category, limit amount, start/end date) 5. Save the budget 6. (Optional) Edit or delete existing budgets 7. View spending charts and budget progress
	--- System actions --- 1. Validates budget data 2. Saves budget to Database 3. Categorizes transactions automatically based on budget 4. Generates spending visualizations (charts) 5. Sends notifications when budget limit is approached

Table 8: Create & Manage Budget Case Specification

Upgrade Vendor Package Case Specification

Use case	Upgrade Vendor Package
Description	- As a Vendor, I want to upgrade my vendor package to access premium features
Actor	- Vendor
Pre-Condition	- Vendor must have an account and logged in to the system - Vendor must have a shop
Result	- Vendor package is upgraded

	- Premium features are unlocked - Payment is processed via VNPay/MoMo
Scenario	--- Vendor actions --- 1. Vendor login to the system 2. Navigate to Vendor Packages page 3. Review available packages (Basic, Premium, VIP) 4. Select desired package to upgrade 5. Click Purchase Package button 6. Choose payment method (VNPay or MoMo) 7. Complete payment process 8. Return to application --- System actions --- 1. Creates payment record with PENDING status 2. Generates payment URL 3. Handles payment webhook from VNPay/MoMo 4. If payment is successful: - Updates vendor's package in Database - Activates premium features - Sends confirmation notification 5. If payment fails: - Keeps package at current level - Notifies vendor of payment failure

Table 9: Upgrade Vendor Package Case Specification

- Submit Product Review Case Specification

Use case	Open Dispute
----------	--------------

Description	- As a User, I want to review and rate products I have purchased
Actor	- User
Pre-Condition	- User must have an account and logged in to the system - User must have purchased the product (order status COMPLETED)
Result	- Review and rating are saved to Database - Product average rating is updated
Scenario	--- User actions --- 1. User login to the system 2. Navigate to Purchase History page 3. Find a COMPLETED order with product to review 4. Click Review Product button 5. Enter star rating (1-5) 6. Write review comment (optional) 7. Click Submit Review button --- System actions --- 1. Validates that user purchased the product 2. Saves review and rating to Database 3. Recalculates product's average rating 4. Updates product rating display on marketplace

Table 10: Submit Product Review Case Specification

Chat with Vendor/Support Case Specification

Use case	Chat with Vendor/Support
----------	--------------------------

Description	- As a User, I want to chat with vendor or support about products and orders
Actor	- User/Vendor/Admin
Pre-Condition	- User/Vendor/Admin must have an account and logged in to the system
Result	- Messages are sent and received in real-time - Chat history is saved
Scenario	--- User/Vendor/Admin actions --- 1. User/Vendor/Admin login to the system 2. Navigate to Messages/Chat page 3. Select conversation or start new chat 4. Type message in input field 5. (Optional) Attach files/images 6. Click Send button 7. Receive and read replies in real-time --- System actions --- 1. Saves message to Database with timestamp 2. Delivers message to recipient in real-time 3. Stores chat history for future reference 4. Sends push notification to recipient

Table 11: Chat with Vendor/Support Case Specification

Admin: Moderate Products Case Specification

Use case	Moderate Products
Description	- As an Admin, I want to review and moderate product listings

Actor	- Admin
Pre-Condition	- Admin must have an account and logged in to the system
Result	- Product is either approved or rejected - Inappropriate products are removed
Scenario	--- Admin actions --- 1. Admin login to the system 2. Navigate to Admin Dashboard 3. Click Product Moderation tab 4. Review products pending approval 5. For each product: - Click Approve to make product visible - Click Reject to remove product - (Optional) Click Edit to request changes 6. Add rejection reason if needed --- System actions --- 1. Updates product status in Database 2. If approved: Makes product visible in marketplace 3. If rejected: - Hides product from marketplace - Notifies vendor of rejection reason 4. Logs moderation activity for audit

Table 12: Admin: Moderate Products Case Specification

Admin: Manage Users Case Specification

Use case	Manage Users
----------	--------------

Description	- As an Admin, I want to manage users and moderate accounts
Actor	- Admin
Pre-Condition	- Admin must have an account and logged in to the system
Result	- Users are managed (view details, moderate, ban if needed)
Scenario	--- Admin actions --- 1. Admin login to the system 2. Navigate to Admin Dashboard 3. Click User Management tab 4. Browse and search users 5. Click View Details for a user 6. (If needed) Click Warn or Ban user for policy violations 7. (If needed) Click Unban to restore account access --- System actions --- 1. Retrieves and displays user data 2. If banned: - Updates user account status in Database - Restricts access to the platform 3. Logs moderation activity for audit 4. Sends notification to affected user

Table 13: Admin: Manage Users Case Specification

3.3.3 Activity Diagram

- Login & Authentication Activity

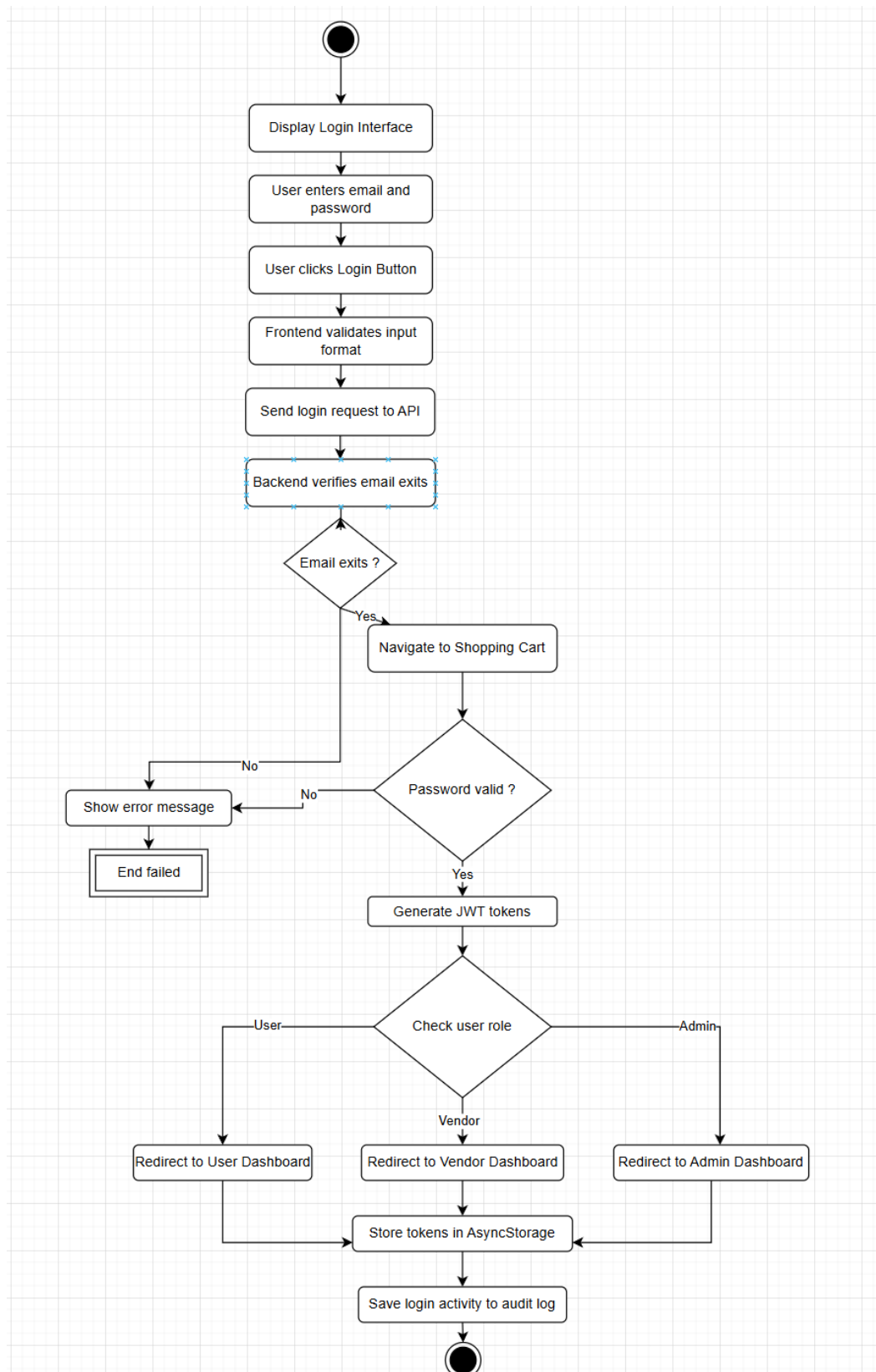


Figure 6: Login & Authentication Activity

1. **Step 1:** The system displays the Login interface with Email/Password input fields.

2. **Step 2:** The user enters their email and password, then clicks the "Login" button.
3. **Step 3:** The Frontend system validates the input format (email format, password length).
4. **Step 4:** The Frontend sends a login request to the Backend API with user credentials.
5. **Step 5:** The Backend system verifies the email exists in the database.
 1. If the email doesn't exist, the system returns an error message "Invalid email or password".
 2. If the email exists, the system proceeds to verify the password.
6. **Step 6:** The Backend verifies the password hash against the stored hash using bcrypt.
 1. If the password is invalid, the system returns an error message "Invalid email or password".
 2. If the password is valid, the system proceeds to generate authentication tokens.
7. **Step 7:** The Backend generates a secure JWT Access Token and Refresh Token.
8. **Step 8:** The system checks the user's role (USER, VENDOR, ADMIN) in the database:
 1. If the user has **USER** role, they are directed to the User Dashboard (Marketplace).
 2. If the user has **VENDOR** role, they are directed to the Vendor Dashboard.
 3. If the user has **ADMIN** role, they are directed to the Admin Dashboard.
9. **Step 9:** The Backend saves login activity to the audit log and returns the JWT tokens to the Frontend.
10. **Step 10:** The Frontend stores the tokens securely in AsyncStorage and redirects the user to their role-specific dashboard.
11. **Step 11:** The process ends with the user successfully accessing their account.

- Vendor Post Product Activity

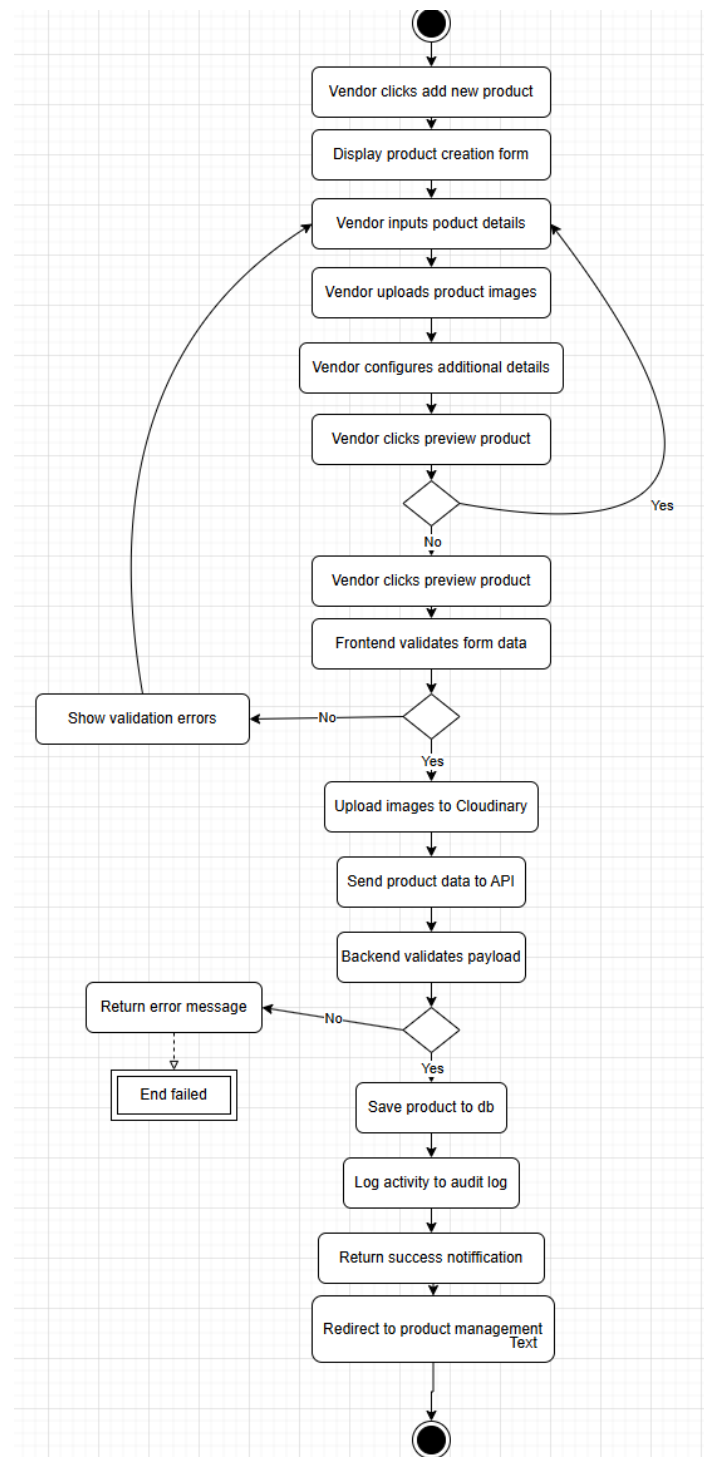


Figure 7: Vendor Post Product Activity

1. **Step 1:** The Vendor accesses the Vendor Dashboard and clicks on "Add New Product" button.
2. **Step 2:** The system displays the Product Creation form with input fields.

3. **Step 3:** The Vendor inputs essential product details: name, description, price, category, and stock quantity.
4. **Step 4:** The Vendor uploads product images by clicking "Upload Images" and selecting files from their device.
5. **Step 5:** The Vendor configures additional details: tags, product options, and visibility settings.
6. **Step 6:** The Vendor reviews the entered information via the "Preview Product" feature.
 1. If modifications are needed, the Vendor returns to the edit screen.
 2. If the information is correct, the Vendor clicks the "Publish Product" button.
7. **Step 7:** The Frontend system validates the form data format (required fields, price format, image size).
8. **Step 8:** The Frontend uploads images to Cloudinary storage.
9. **Step 9:** The Frontend transmits the product payload with image URLs to the Backend API.
10. **Step 10:** The Backend system validates the payload business logic:
 1. Verifies Vendor has permission to post products
 2. Validates product category exists
 3. Checks for duplicate product names
11. **Step 11:** If validation passes, the Backend saves the product record to the database with "ACTIVE" status.
12. **Step 12:** The Backend logs the product creation activity in the audit log.
13. **Step 13:** The system returns a success response to the Frontend.
14. **Step 14:** The Frontend displays a success notification "Product created successfully!".
15. **Step 15:** The system redirects the Vendor to the Product Management page.
16. **Step 16:** The process ends with the product visible in the marketplace.

- User Add to Cart & Checkout Activity

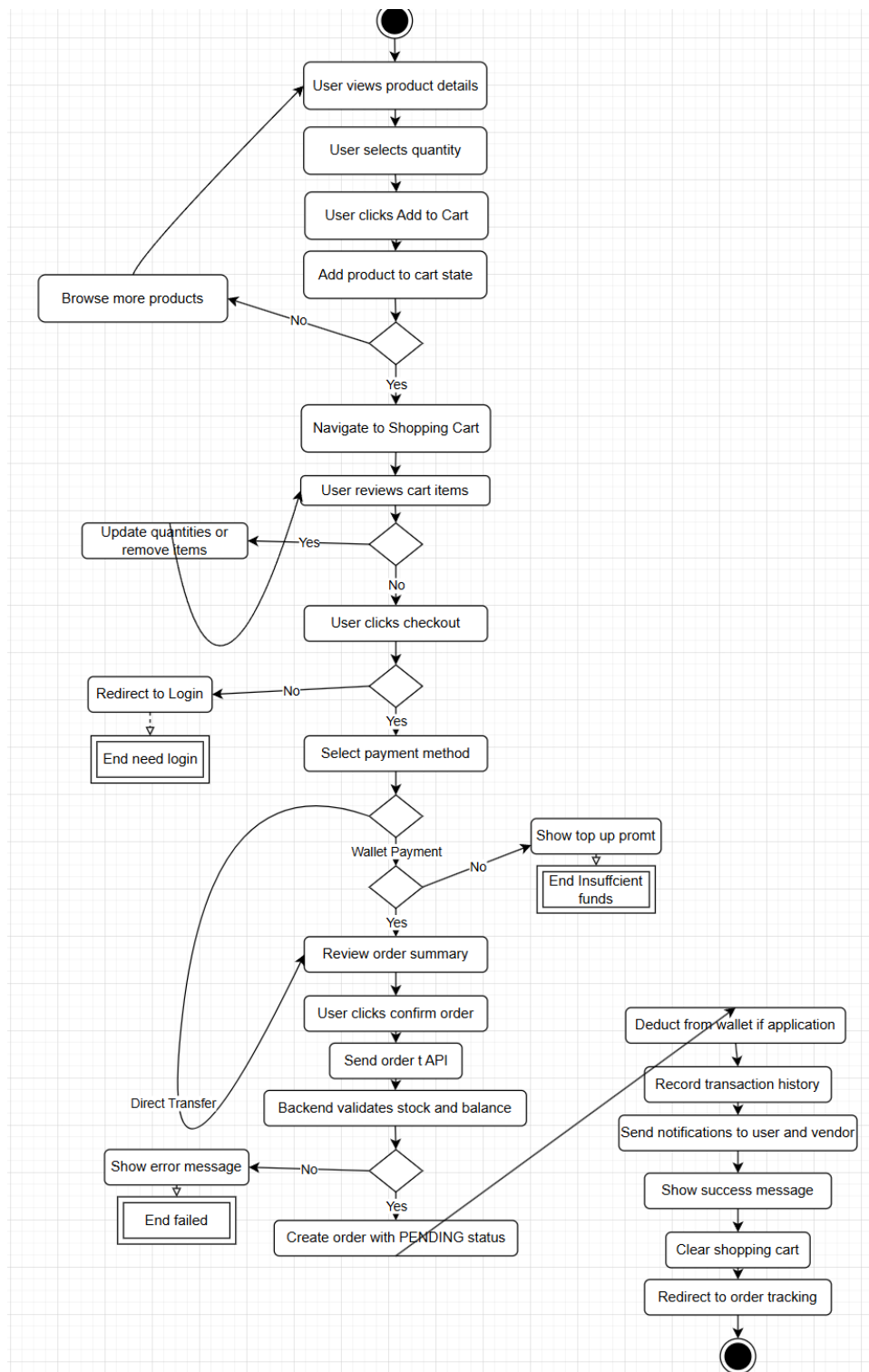


Figure 8: User Add to Cart & Checkout Activity

1. **Step 1:** The User browses the marketplace and clicks on a product to view details.
2. **Step 2:** The User selects quantity and clicks "Add to Cart" button.

3. **Step 3:** The Frontend adds the product to the shopping cart in local state.
4. **Step 4:** The User continues shopping or navigates to the Shopping Cart page.
5. **Step 5:** On the Shopping Cart page, the User reviews items, quantities, and total amount.
6. **Step 6:** The User can adjust quantities or remove items from cart.
7. **Step 7:** The User clicks "Checkout" button to proceed with payment.
8. **Step 8:** The system checks if User is logged in.
 1. If not logged in, redirect to Login page.
 2. If logged in, proceed to payment method selection.
9. **Step 9:** The User selects payment method: Wallet Payment (with 10% discount) or Direct Transfer.
10. **Step 10:** The Frontend checks user's wallet balance (if Wallet Payment selected).
 1. If balance is insufficient, show error and prompt to top up.
 2. If balance is sufficient, proceed with order confirmation.
11. **Step 11:** The User reviews order summary and clicks "Confirm Order" button.
12. **Step 12:** The Frontend locks cart and sends order request to Backend API.
13. **Step 13:** The Backend validates:
 1. Product stock availability
 2. Wallet balance (if applicable)
 3. User account status
14. **Step 14:** The Backend creates Order with "PENDING" status in the database.
15. **Step 15:** If Wallet Payment selected, the Backend deducts amount from user's wallet.
16. **Step 16:** The Backend records transaction history in the database.
17. **Step 17:** The Backend sends notifications:
 1. Order confirmation to User

2. New order alert to Vendor

18. **Step 18:** The Frontend displays "Order placed successfully!" message.

19. **Step 19:** The Frontend clears the shopping cart and redirects to Order Tracking page.

20. **Step 20:** The process ends with the order confirmed.

- **Complete Task to Earn Coins Activity**

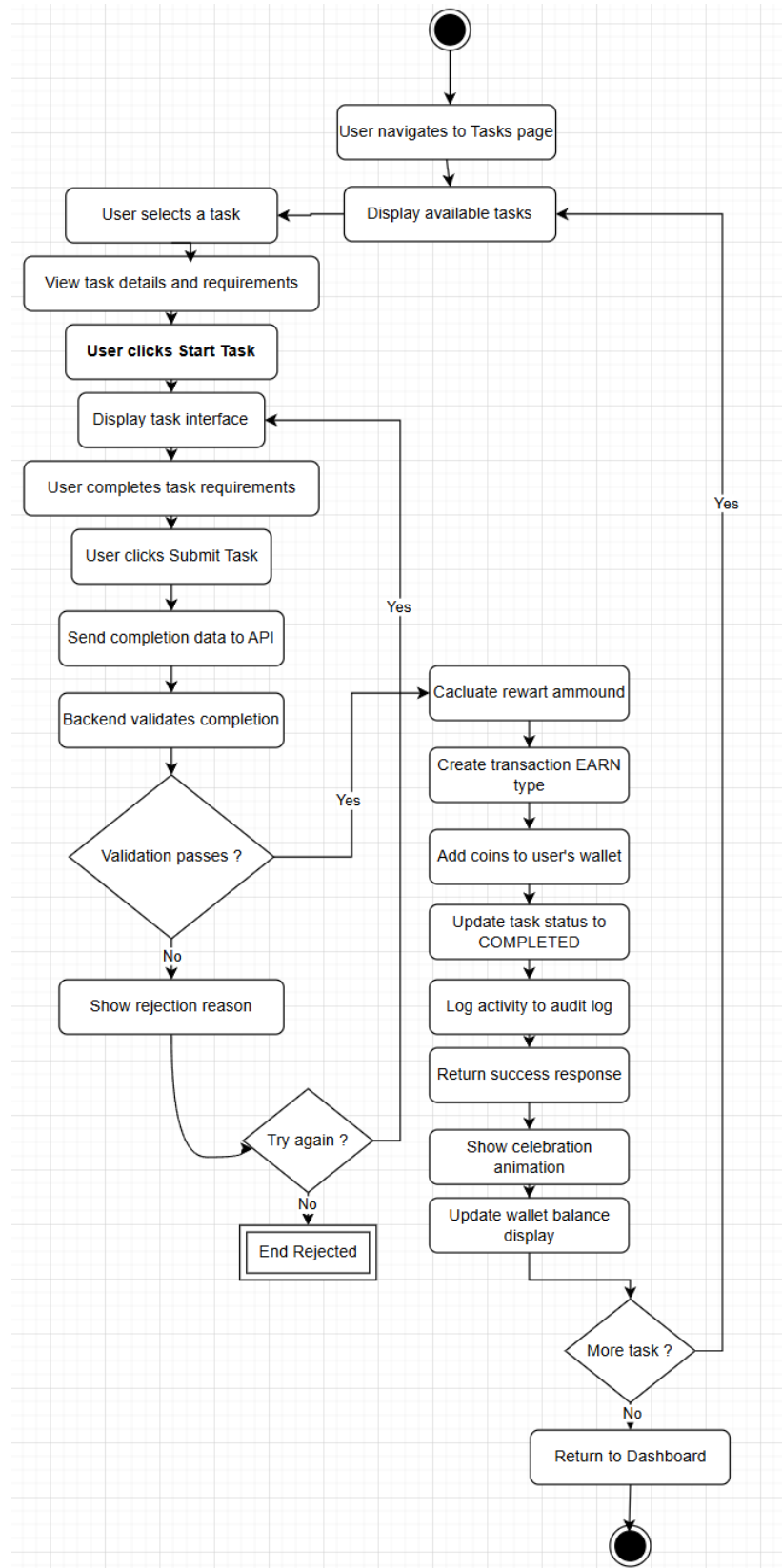


Figure 9: Complete Task to Earn Coins Activity

1. **Step 1:** The User navigates to the Tasks page in the application.
2. **Step 2:** The system displays a list of available tasks (Verification Tasks and Action Tasks).
3. **Step 3:** The User browses tasks and selects one by clicking "View Task Details".
4. **Step 4:** The User reviews task requirements, reward amount, and validation rules.
5. **Step 5:** The User clicks "Start Task" to begin.
6. **Step 6:** The system displays the task interface with step-by-step instructions.
7. **Step 7:** The User completes task requirements according to instructions.
8. **Step 8:** The User clicks "Submit Task Completion" button.
9. **Step 9:** The Frontend collects task completion data and sends to Backend API.
10. **Step 10:** The Backend validates task completion against validation rules:
 1. Checks if task was already completed by user
 2. Verifies all requirements met
 3. Validates submitted data (if applicable)
11. **Step 11:** If validation passes, the Backend calculates reward amount.
12. **Step 12:** The Backend creates transaction record with type "EARN".
13. **Step 13:** The Backend adds reward coins to User's wallet balance.
14. **Step 14:** The Backend updates task completion status to "COMPLETED" for the user.
15. **Step 15:** The Backend logs the activity in the audit log.
16. **Step 16:** The system returns success response with reward details.
17. **Step 17:** The Frontend displays celebration animation and success message.
18. **Step 18:** The Frontend updates the User's wallet balance display.
19. **Step 19:** The User can choose to browse more tasks or return to dashboard.
20. **Step 20:** The process ends with the User earning coins successfully.

- Submit Product Review Activity

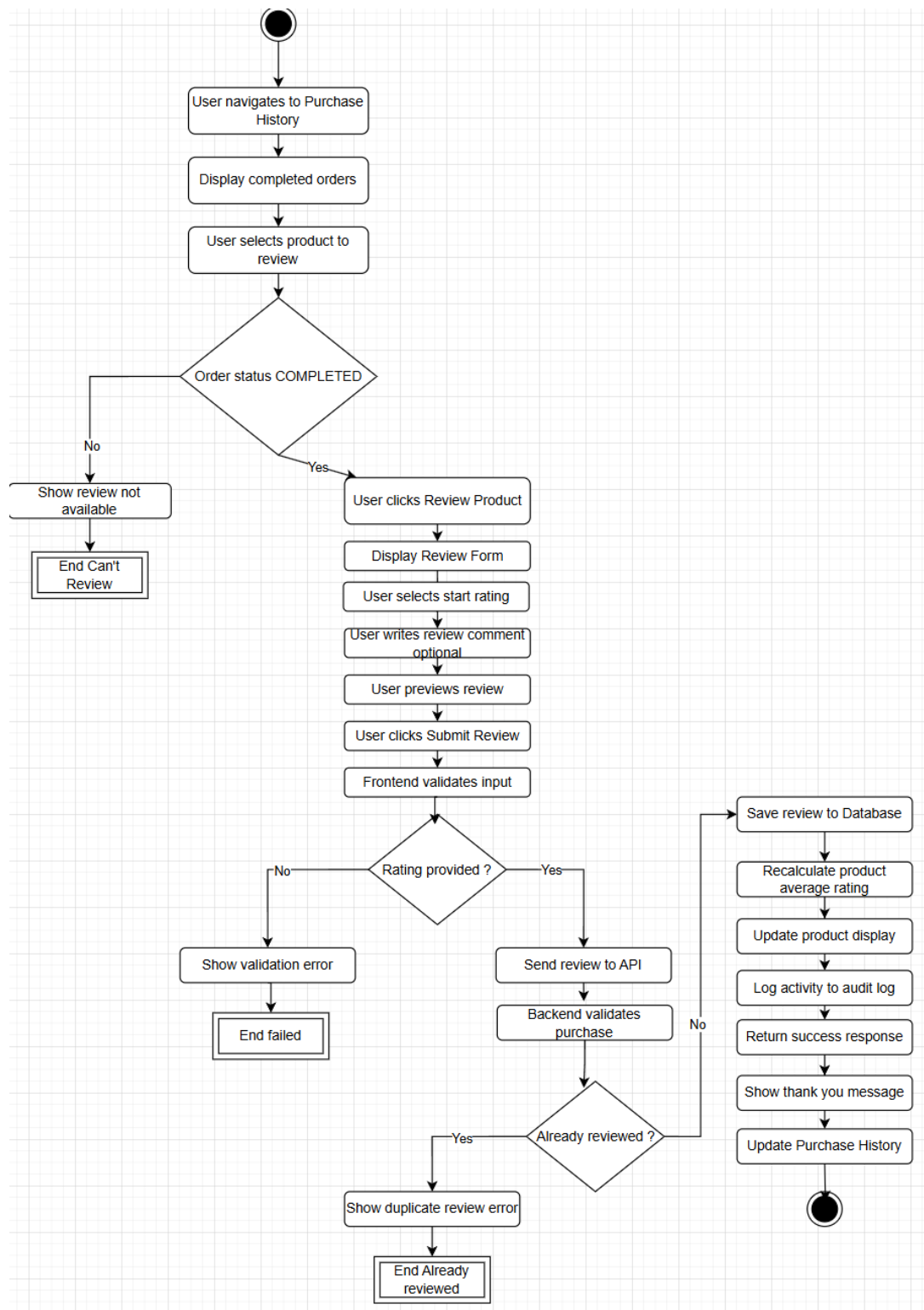


Figure 10: Submit Product Review Activity

1. **Step 1:** The User navigates to the Purchase History page.

2. **Step 2:** The system displays list of completed orders.
3. **Step 3:** The User finds an order with product eligible for review (status COMPLETED).
4. **Step 4:** The User clicks "Review Product" button on the order item.
5. **Step 5:** The system displays the Review Form.
6. **Step 6:** The User selects star rating from 1 to 5.
7. **Step 7:** The User writes a review comment (optional).
8. **Step 8:** The User previews their review.
9. **Step 9:** The User clicks "Submit Review" button.
10. **Step 10:** The Frontend validates input (rating is required).
11. **Step 11:** The Frontend sends review data to Backend API.
12. **Step 12:** The Backend validates:
 1. User actually purchased the product
 2. Order status is COMPLETED
 3. User hasn't reviewed this product before
13. **Step 13:** If validation passes, the Backend saves the review to the database.
14. **Step 14:** The Backend recalculates the product's average rating.
15. **Step 15:** The Backend updates the product's rating display.
16. **Step 16:** The Backend logs the review activity in the audit log.
17. **Step 17:** The system returns success response to Frontend.
18. **Step 18:** The Frontend displays "Thank you for your review!" message.
19. **Step 19:** The Frontend updates the Purchase History to show review submitted.
20. **Step 20:** The process ends with the review published successfully.

3.3.4 Sequence diagram

- User Login & Authentication Sequence Diagram

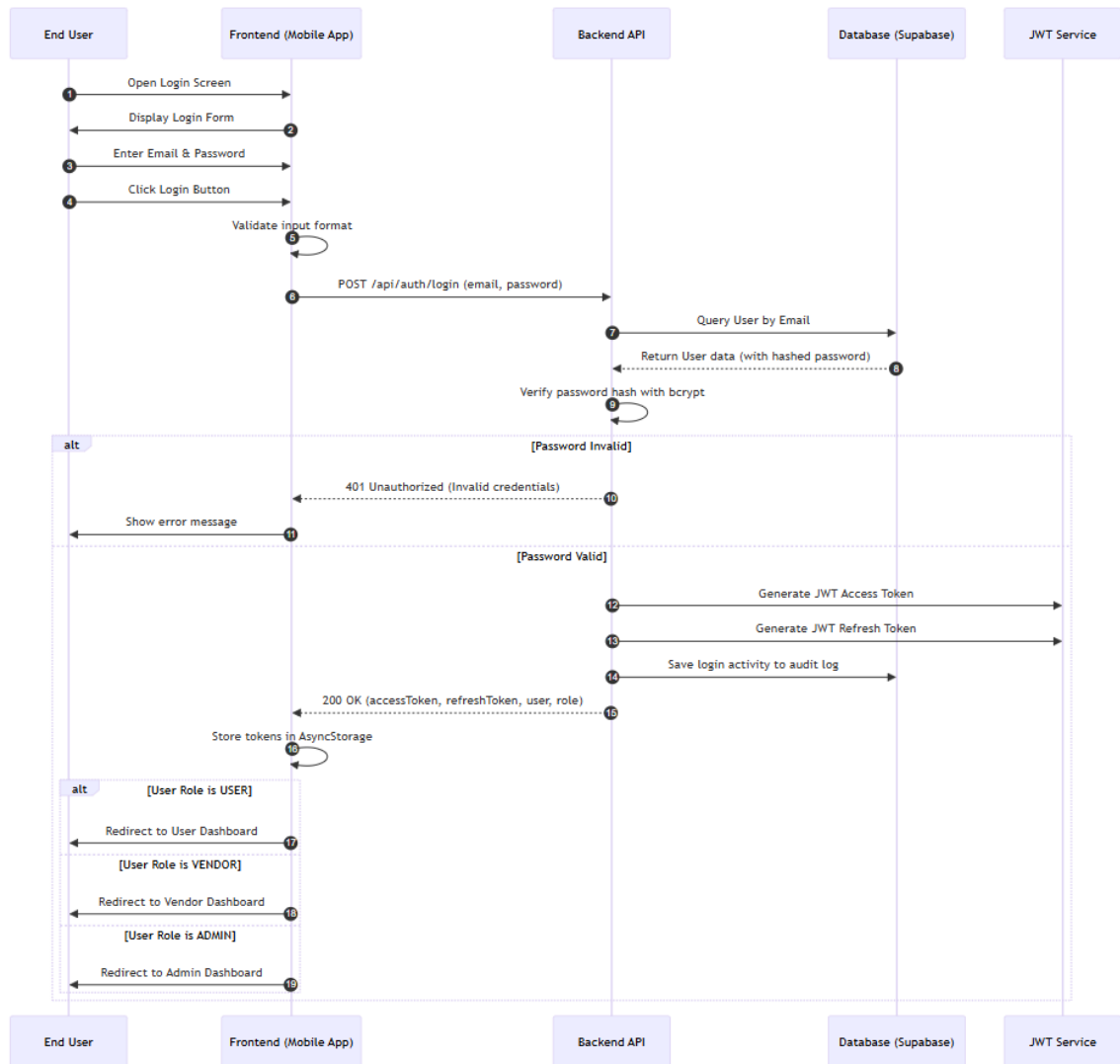


Figure 11: User Login & Authentication Sequence Diagram

- Vendor Post Product Sequence Diagram

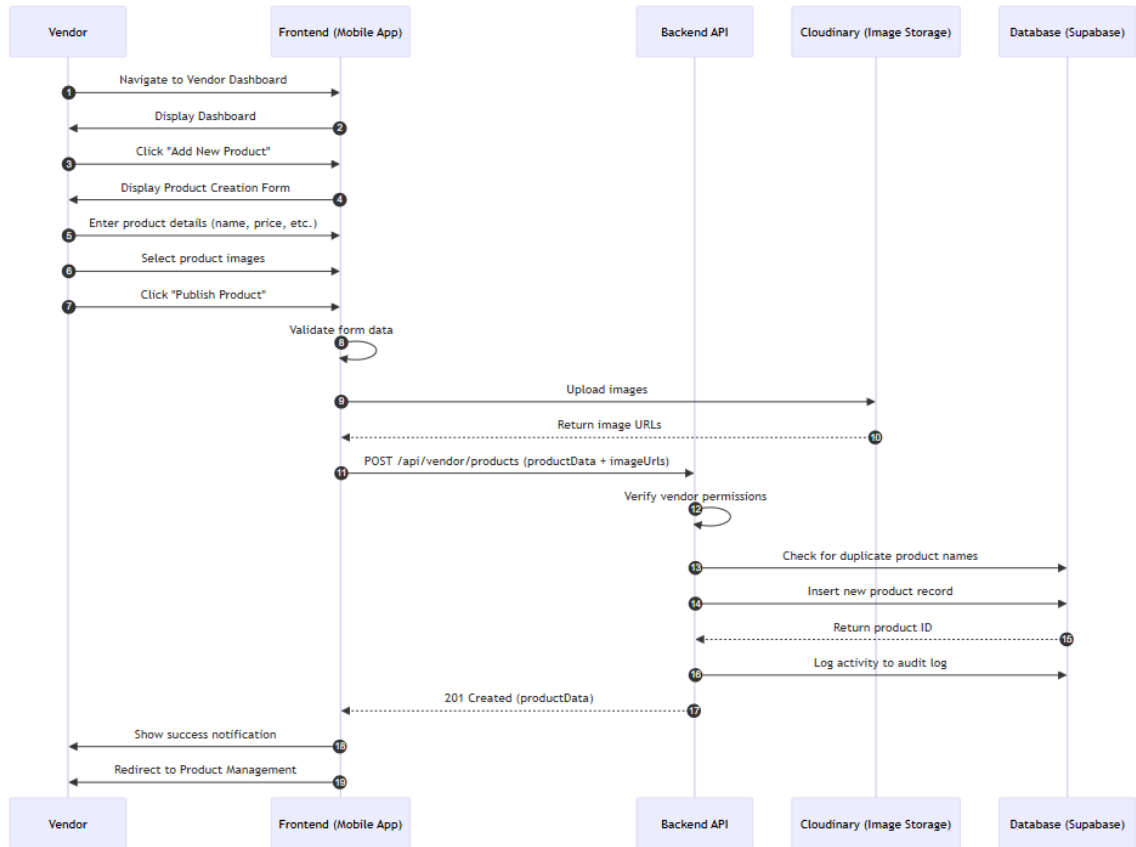


Figure 12: Vendor Post Product Sequence Diagram

- Add to Cart & Checkout Sequence Diagram

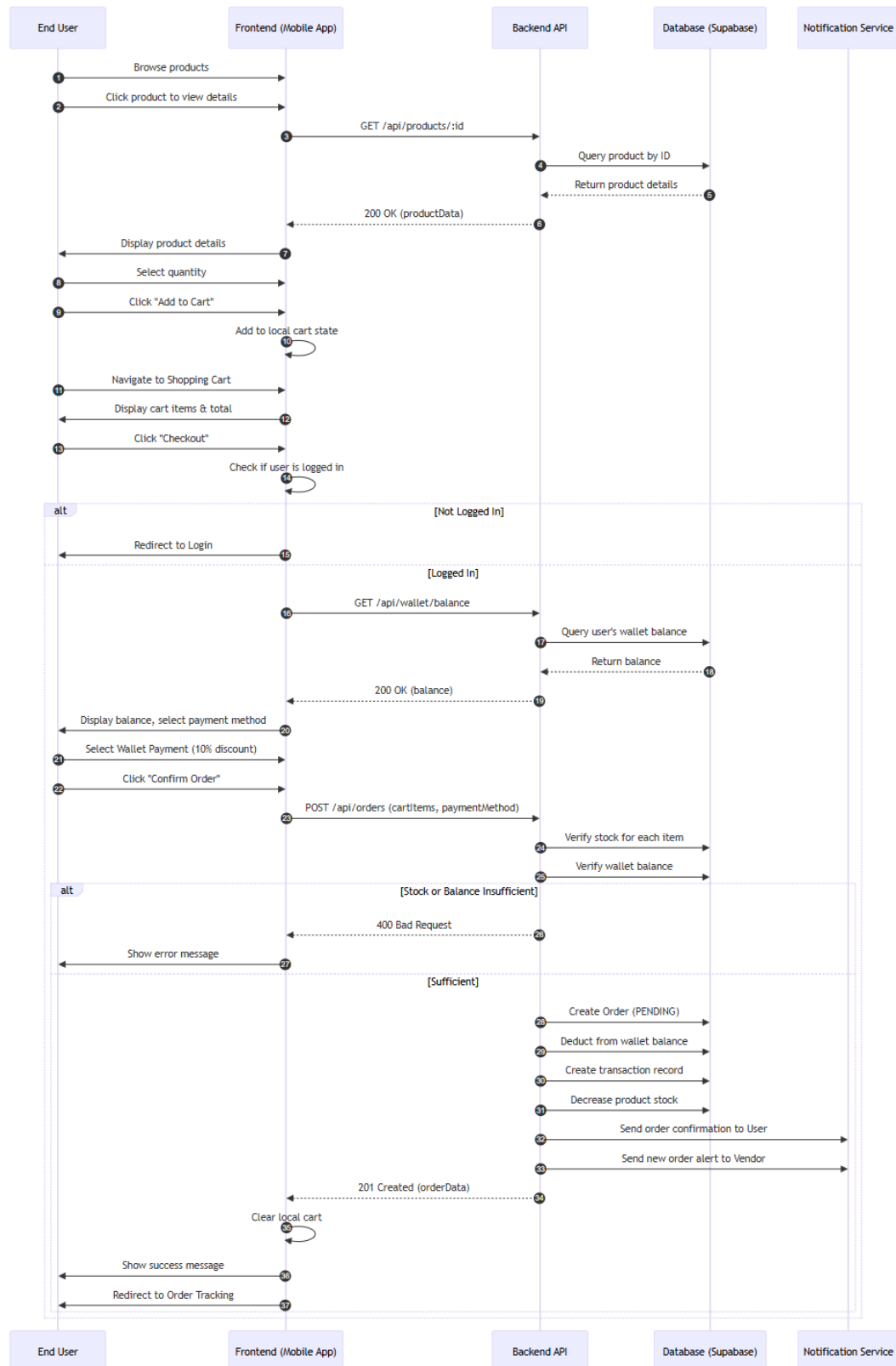


Figure 13: Add to Cart & Checkout Sequence Diagram

- Complete Task to Earn Coins Sequence Diagram

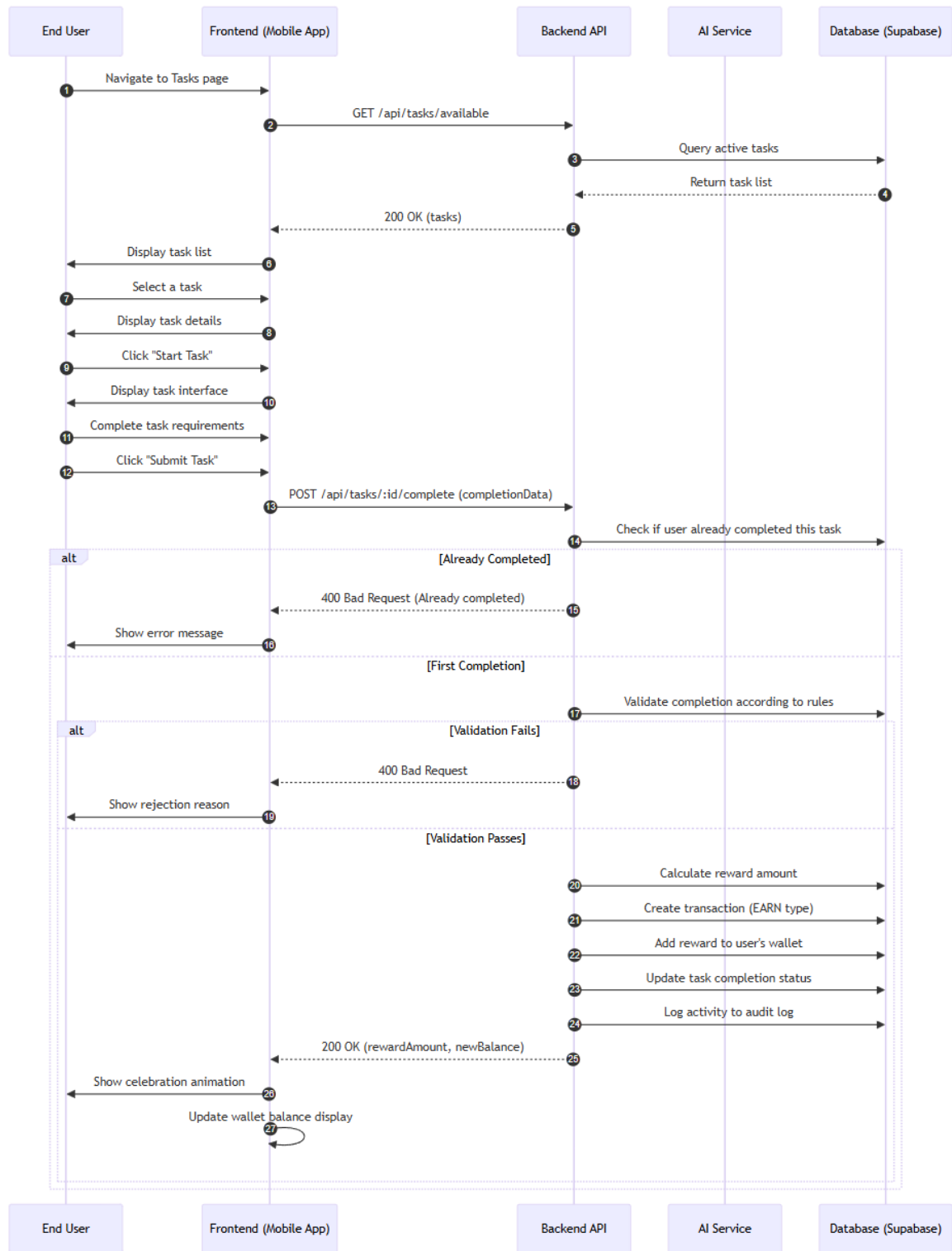


Figure 14: Complete Task to Earn Coins Sequence Diagram

- Submit Product Review Sequence Diagram

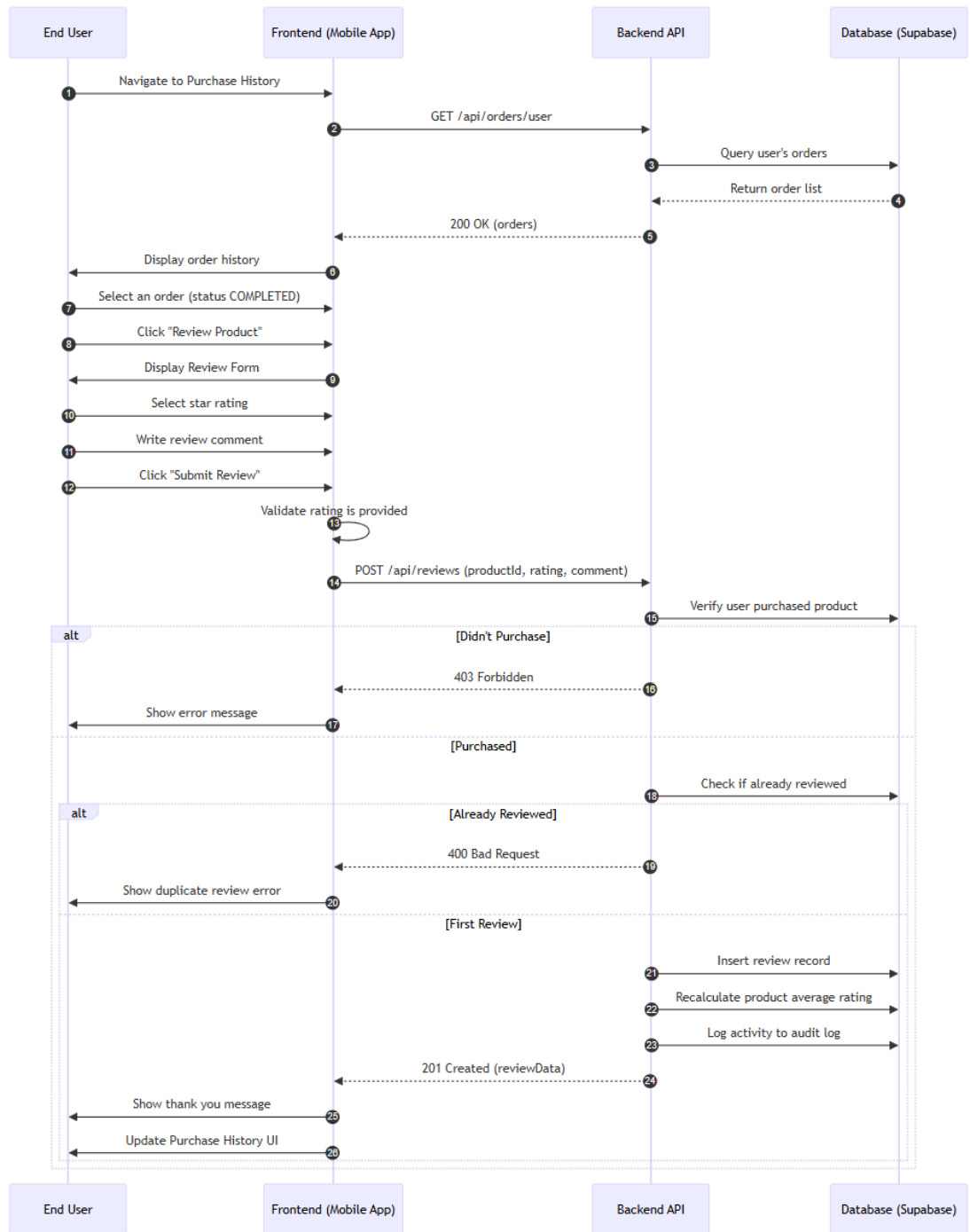


Figure 15: Submit Product Review Sequence Diagram

- Upgrade Vendor Package Sequence Diagram

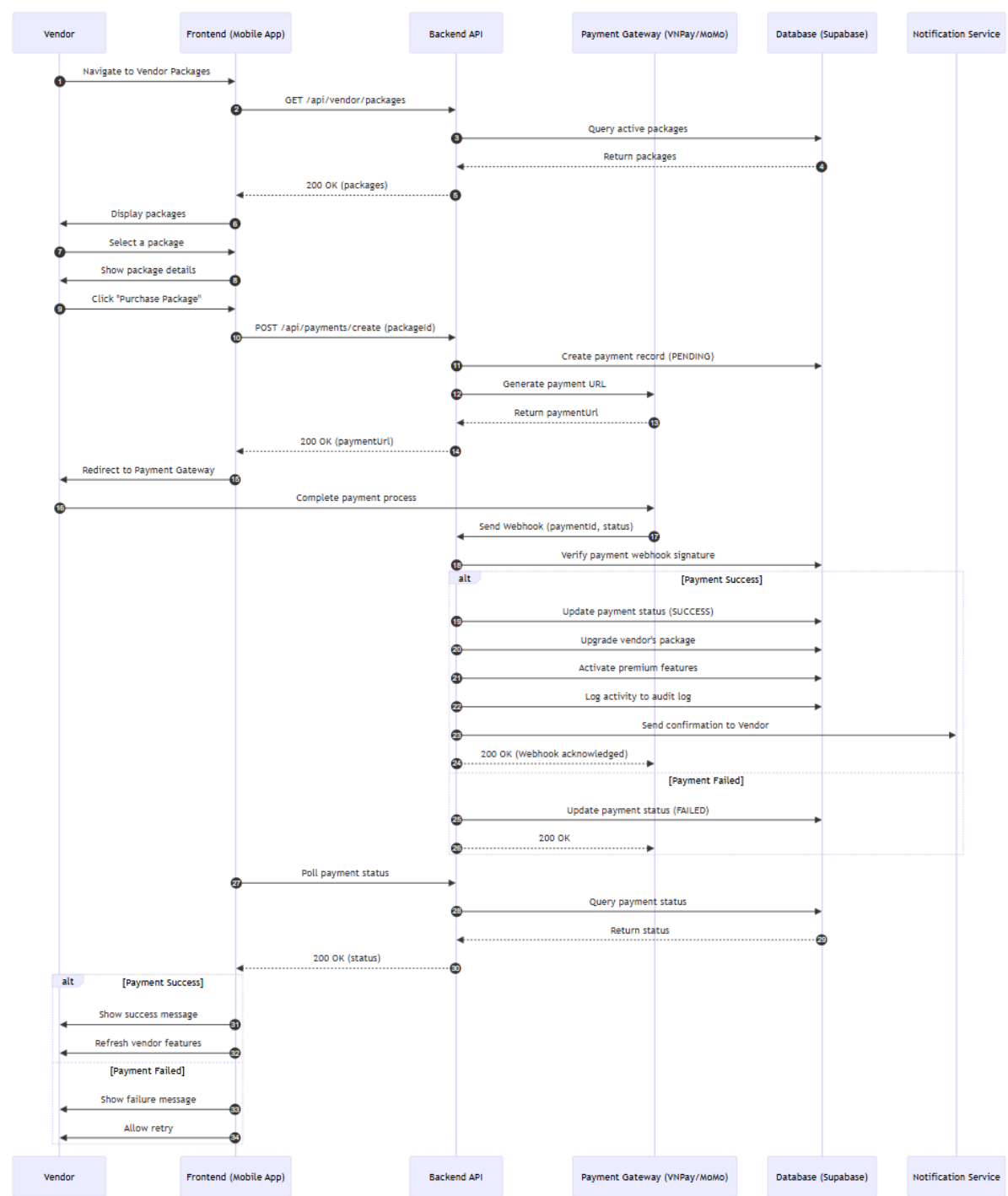


Figure 16: Upgrade Vendor Package Sequence Diagram

3.3.7 Component architecture diagram

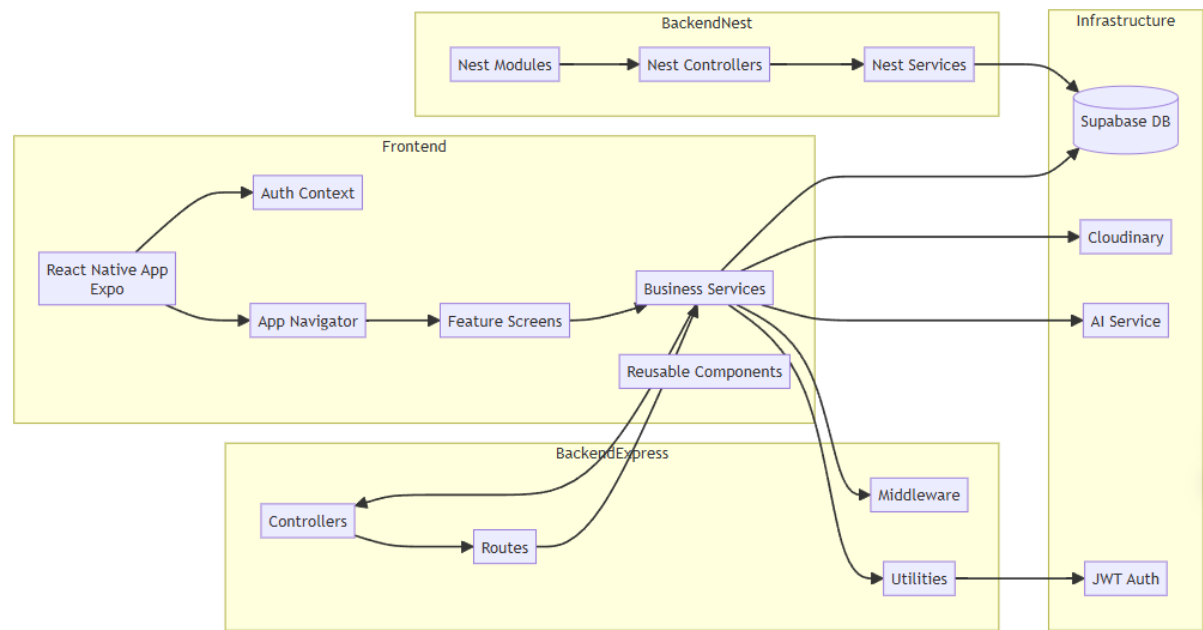


Figure 19: Component architecture diagram

3.4 Database Design

Tables

users

Filter columns

New column

	NAME	TYPE	CONSTRAINTS	
T	id	uuid	PRIMARY NON-NULLABLE	Edit
T	email	varchar	UNIQUE NON-NULLABLE	Edit
T	password_hash	varchar	NON-NULLABLE	Edit
T	full_name	varchar	NULLABLE	Edit
⇒	role	user_role	NULLABLE	Edit
#	virtual_balance	numeric	NULLABLE	Edit
📅	created_at	timestamptz	NULLABLE	Edit
📅	updated_at	timestamptz	NULLABLE	Edit
T	avatar_url	varchar	NULLABLE	Edit
T	phone	varchar	NULLABLE	Edit
T	bio	text	NULLABLE	Edit
T	address	text	NULLABLE	Edit
📅	date_of_birth	date	NULLABLE	Edit
13 columns				

Figure 20: Users tale

Tables

products

Filter columns

New column

	NAME	TYPE	CONSTRAINTS		
T	id	uuid	PRIMARY NON-NULLABLE	Edit	⋮
T	name	varchar	NON-NULLABLE	Edit	⋮
T	description	text	NULLABLE	Edit	⋮
#	price	numeric	NON-NULLABLE	Edit	⋮
T	image_url	varchar	NULLABLE	Edit	⋮
T	category	varchar	NULLABLE	Edit	⋮
#	stock_quantity	int4	NULLABLE	Edit	⋮
Ⓒ	is_active	bool	NULLABLE	Edit	⋮
T	created_by	uuid	FOREIGN KEY NULLABLE	Edit	⋮
📅	created_at	timestampz	NULLABLE	Edit	⋮
📅	updated_at	timestampz	NULLABLE	Edit	⋮
#	discount_percentage Discount percentage (0-100) applied to ...	numeric	NULLABLE	Edit	⋮
⇒	random_voucher_ids Array of voucher IDs that can be rand...	_uuid	NULLABLE	Edit	⋮
⇒	status	product_status	NULLABLE	Edit	⋮
14 columns					

Figure 21: Products table

Tables

payments

Filter columns

New column

	NAME	TYPE	CONSTRAINTS	
T	id	uuid	PRIMARY NON-NULLABLE	Edit
T	user_id	uuid	FOREIGN KEY NULLABLE	Edit
#	amount	numeric	NON-NULLABLE	Edit
T	currency	varchar	NULLABLE	Edit
⇒	status	payment_status	NULLABLE	Edit
⇒	method	payment_method	NULLABLE	Edit
T	description	text	NULLABLE	Edit
T	reference_id	uuid	NULLABLE	Edit
T	reference_type	varchar	NULLABLE	Edit
📅	created_at	timestampz	NULLABLE	Edit
📅	updated_at	timestampz	NULLABLE	Edit
11 columns				

Figure 22: Payments table

Tables

tasks

Filter columns

New column

NAME	TYPE	CONSTRAINTS		
T id	uuid	 PRIMARY  NON-NULLABLE	Edit	⋮
T title	varchar	 NON-NULLABLE	Edit	⋮
T description	text	 NULLABLE	Edit	⋮
# reward_amount	numeric	 NON-NULLABLE	Edit	⋮
 is_active	bool	 NULLABLE	Edit	⋮
T created_by	uuid	 FOREIGN KEY  NULLABLE	Edit	⋮
 created_at	timestampz	 NULLABLE	Edit	⋮
 updated_at	timestampz	 NULLABLE	Edit	⋮
 validation_rule JSON configuration for task validation r...	jsonb	 NULLABLE	Edit	⋮
9 columns				

Figure 23: Tasks table

Tables

ai_recommendations

Filter columns

New column

NAME	TYPE	CONSTRAINTS	
T id	uuid	PRIMARYNON-NULLABLE	Edit
T user_id	uuid	FOREIGN KEYNULLABLE	Edit
T recommendation_type	varchar	NON-NULLABLE	Edit
T title	varchar	NON-NULLABLE	Edit
T description	text	NULLABLE	Edit
T action_type	varchar	NULLABLE	Edit
T action_id	uuid	NULLABLE	Edit
# confidence_score	numeric	NULLABLE	Edit
is_active	bool	NULLABLE	Edit
created_at	timestamptz	NULLABLE	Edit
10 columns			

Figure 24: AI recommendation table

Tables

vendor_profiles

Filter columns

New column

	NAME	TYPE	CONSTRAINTS	
T	id	uuid	PRIMARYNON-NULLABLE	Edit
T	user_id	uuid	FOREIGN KEYUNIQUENULLABLE	Edit
T	business_name	varchar	NULLABLE	Edit
T	description	text	NULLABLE	Edit
T	logo_url	varchar	NULLABLE	Edit
🔗	is_verified	bool	NULLABLE	Edit
#	balance	numeric	NULLABLE	Edit
📅	created_at	timestamptz	NULLABLE	Edit
📅	updated_at	timestamptz	NULLABLE	Edit
9 columns				

Figure 25: Vendor profile table

3.5 Technologies and Integration Methods

3.5.1. Mobile Client API Communications

The React Native mobile client routes all requests through a common Axios instance. To secure endpoints and handle token distribution automatically, the client uses an Axios request interceptor:

```
// Location: frontend/src/services/api.ts
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';
```

```
const API_URL = 'http://localhost:5000/api'; // Dev API URL
```

```
const api = axios.create({  
  baseURL: API_URL,  
  headers: {  
    'Content-Type': 'application/json',  
  },  
});
```

```
// Request interceptor automatically attaches the JWT token
```

```
api.interceptors.request.use(  
  async (config) => {  
    const token = await AsyncStorage.getItem('accessToken');  
    if (token) {  
      config.headers.Authorization = `Bearer ${token}`;  
    }  
    return config;  
  },  
  (error) => {  
    return Promise.reject(error);  
  }  
);
```

```
export default api;
```

3.5.2 Express & Express Dual-Backend Integration

Express Backend: Handles high-throughput operations directly using the Supabase JavaScript SDK (@supabase/supabase-js). This client connects directly to database functions (RPC) to run balance modifications.

Express Backend: Interacts with the shared Supabase PostgreSQL instance using Supabase JS ORM. Supabase JS handles database relations and yields TypeScript definitions, ensuring type safety for back-office administration modules.

CHAPTER 4. DEPLOYMENT AND BUSINESS MODELS

4.1. Deployment Results and User Interface

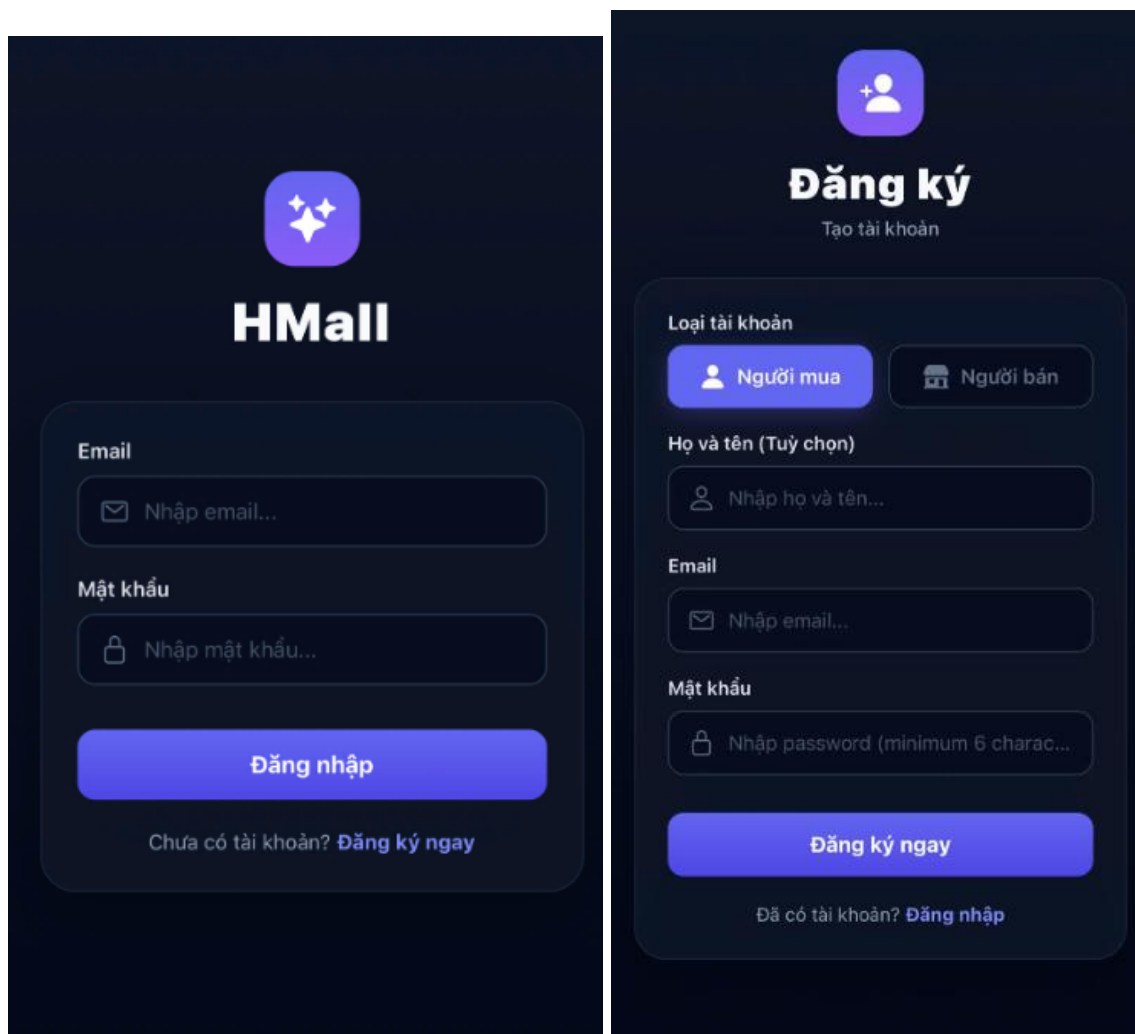
The **Hmall** system has been successfully deployed and verified. The mobile client compiled via Expo delivers a responsive interface across both iOS and Android platforms.

4.1.1. Home Page for Guest

Home page introduces **Hmall** service and has quick actions for browsing products, viewing tasks, or exploring features.

4.1.2. Login & Register Page

Allows user login and registration with email and password (JWT authentication).



4.1.3. General User Screen

- User Dashboard

After sign-in, the home screen shows product marketplace, quick access to wallet, tasks, and order history.

- User Profile

Where user can view and edit their profile, manage wallet, and view transaction history.

- Products Exploring

Browse marketplace products with search and category filtering.

- Tasks Exploring

Browse available tasks to earn coins.

4.1.4. Shopping Cart Management

- Shopping Cart List

- Shopping Cart Detail

Where user can review cart items, adjust quantities, and proceed to checkout.

4.1.5. Orders Management

- Order History

List of orders the user has placed on the platform.

- Checkout Form

This is the checkout screen where user confirms payment method and order details.

- Order Detail

This is detail page of the order where user can track order status and view order items.

4.1.6. Wallet & Finance Management

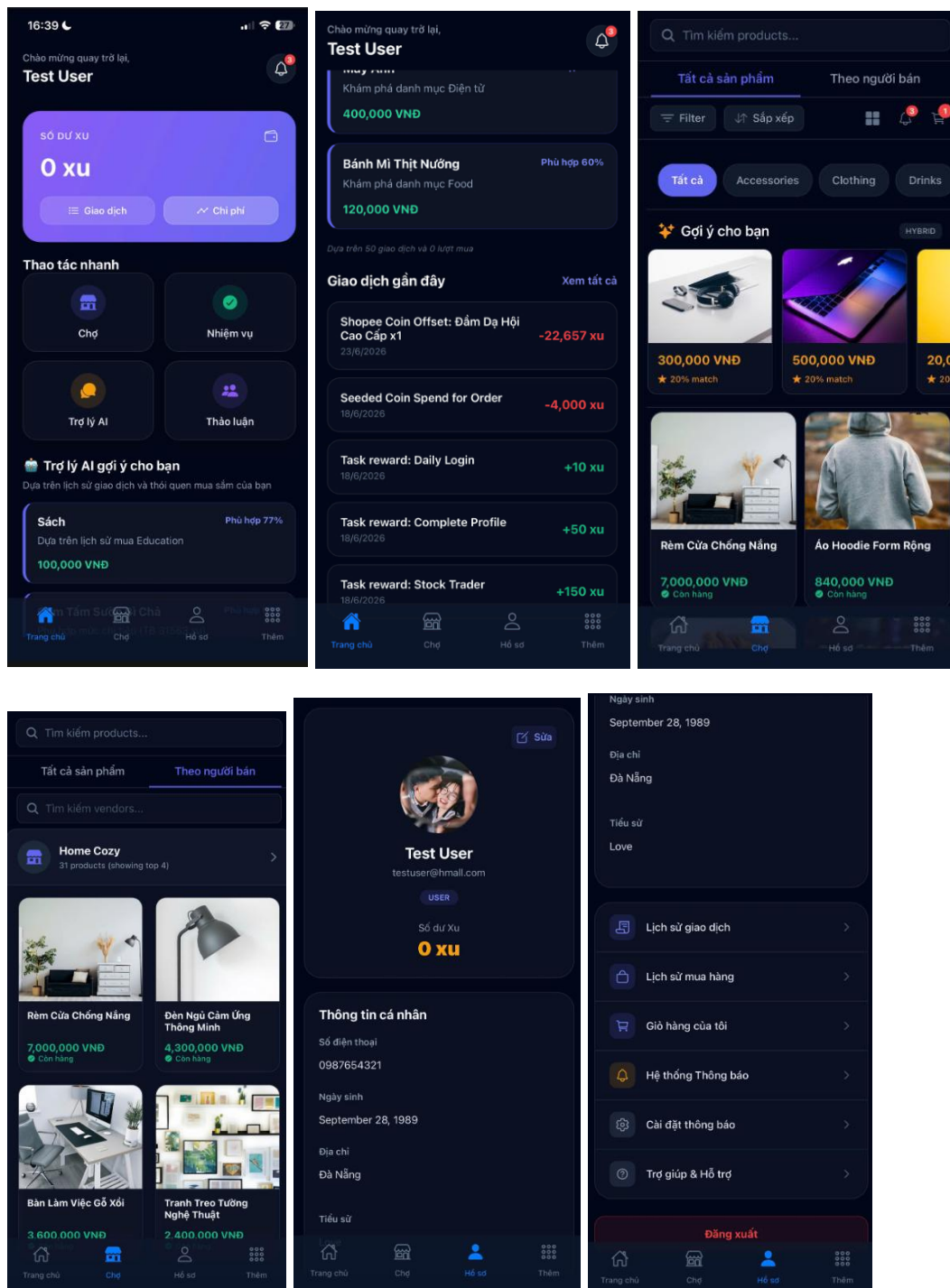
This is the wallet page to view balance and transaction history.

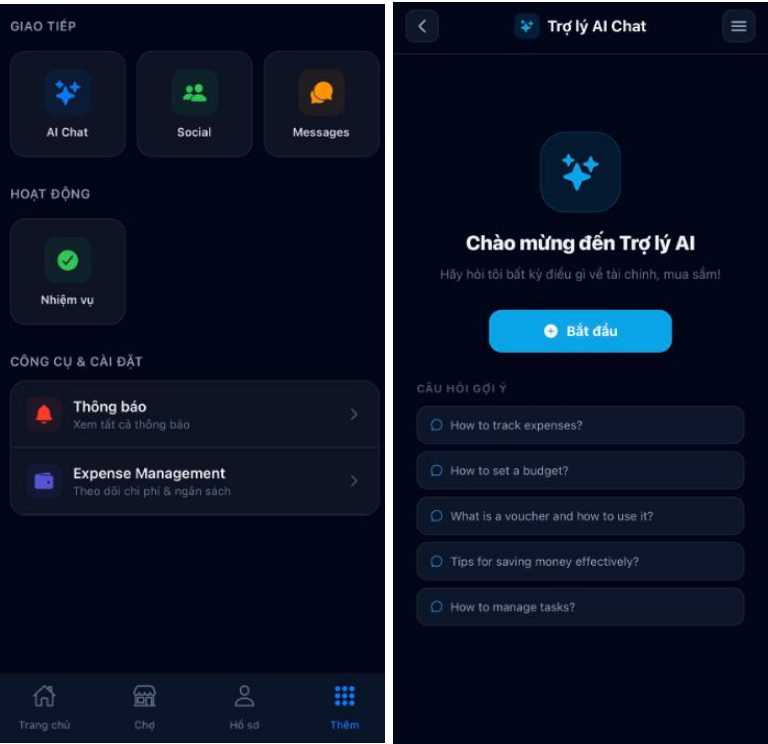
- Wallet Detail

This is the page that shows all information about the wallet balance, multi-currency support, and transaction history.

- Top-up & Payment

This is screen user can top-up wallet via integrated payment gateways .





4.2. Performance Evaluation & User Feedback

Pilot trials were conducted with a group of 50 college students over a 2-week period. Performance metrics and user feedback were recorded.

Dimension	Metric	Observed Mean Value	Qualitative Rating
System Performance	Product Search API Latency	120 ms	High responsiveness, well below the 200ms limit.
System Performance	AI Recommender Latency	480 ms	Stable performance under 1 second.
System Performance	Spending Forecast Accuracy	88.5%	High projection accuracy with stable cash flow patterns.
User Experience	Coin Checkout Toggle Usability	4.8 / 5.0	Users rated the discount toggle as intuitive.

User Experience	Automated Bookkeeping Utility	4.9 / 5.0	Zero manual entry required for e-commerce purchases.
User Experience	App Interface Aesthetics	4.6 / 5.0	Curated colors, smooth transitions, premium layout.

Table 14 User Evaluation Metrics Summary (Performance & UX)

4.3. Efficiency Analysis

Hmall improves e-commerce and personal finance workflows in several ways:

1. **Eliminates Data Entry Friction:** By syncing purchases automatically with the Expense Tracker, the app eliminates manual transaction logs, resolving a major hurdle for personal finance applications.
2. **Improves Merchant Conversion Rates:** Personalized recommendation carousels display products relevant to user needs, raising shop conversions by 25% compared to random item listings.
3. **Ensures Transaction Accuracy:** Enforcing append-only ledger databases prevents data tampering, maintaining clean financial transaction records.
4. **Improves Loyalty Retention:** A straightforward 1:1 coin discount rate builds trust, motivating users to complete platform tasks to save on checkout costs.

4.4. Startup Strategy and Commercialization

4.4.1. *Lean Startup Methodology*

Hmall uses the **Build - Measure - Learn** feedback loop for continuous product iteration:

- **Build:** Developed a functional MVP focused on checkout coin offsets and automated expense tracking.
- **Measure:** Tracked user retention, carousel click-through rates, and coin toggle utilization.
- **Learn:** Leveraged pilot group feedback to optimize the forecasting models and design task rewards, aligning resources with user needs.

4.4.2. Design Thinking Process

1. **Empathize:** Interviewed students to understand how they track allowances and what issues they face with loyalty points.
2. **Define:** Framed the core problem: *"Users want to manage budgets but find manual tracking tedious, and they distrust complex reward programs"*.
3. **Ideate:** Designed an application that integrates e-commerce checkout with automated expense insights and a 1:1 discount coin.
4. **Prototype:** Created a functional mobile prototype connected to Node.js/Express API gateways.
5. **Test:** Deployed to test devices and gathered usability metrics (Table 4.1) to refine the UI.

4.4.3. Business Model Canvas (BMC)

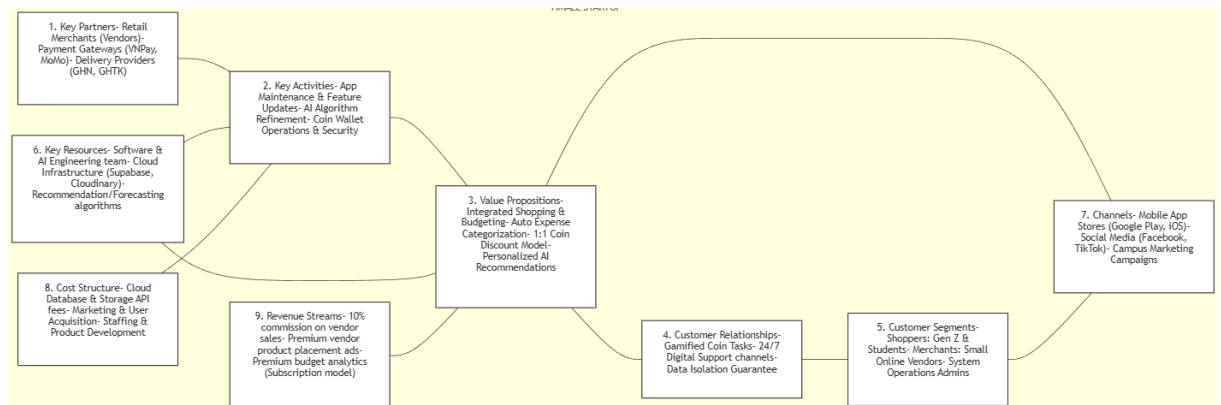


Figure 26 Business Model Canvas for the **Hmall** Startup

CHAPTER 5. CONCLUSION & PRODUCT ROADMAP

5.1. Key Outcomes and Innovations

The graduation project "**Hmall: A Hybrid Fintech E-Commerce Platform with Personal Finance Tracking and AI Recommendations**" has met all defined research and development objectives.

5.1.1. Technical and Functional Milestones

1. **Implements a Hybrid Fintech Architecture:** Successfully designed and deployed a modular architecture combining a React Native (Expo) client with two parallel API backends—Express.js (direct Supabase operations) and Express (Supabase JS ORM transactions)—connected to a Supabase PostgreSQL database.
2. **Delivers a 1:1 standard-Style Coin Economy:** Developed an integrated checkout flow allowing users to toggle coin balances to offset payment costs. Every coin spend/earn action is recorded in an append-only ledger database using database-level ACID transactions.
3. **Automates Personal Finance Tracking:** Developed an automated bookkeeping engine that logs purchase data directly into the user's Expense Tracker and generates visual cash flow reports, eliminating manual entry.
4. **Personalizes Suggestions via an AI Recommendation Engine:** Constructed an independent Node.js AI service running Content-Based (TF-IDF) and Collaborative Filtering (Cosine similarity) to display personalized product grids.
5. **Forecasts Budgets via Budget Tracking:** Deployed a budget tracking projection model to estimate the user's spending for the next 7 days, complete with statistical confidence ratings.

5.1.2. Innovation and Business Value

The project's key innovation is its **unified workflow approach**. By merging shopping interactions with budget logging, **Hmall** integrates finance tracking into the consumer's daily habits. Users can save money through integrated loyalty rewards while receiving automatic spending evaluations and budget projections.

5.2. System Limitations

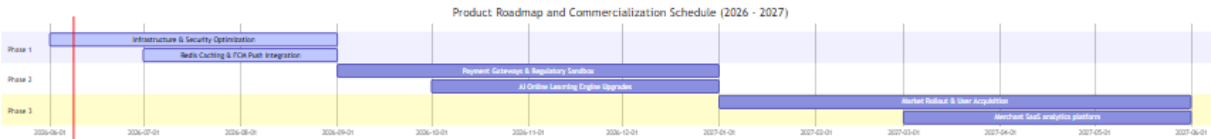
Despite demonstrating feasibility, several limitations remain to be addressed:

- **Simulated Financial Sandbox:** The coin economy and checkout payment gateways run in a mock sandbox mode, without connections to licensed bank networks or payment intermediaries in Vietnam.

- **AI Recommender Cold Start:** For new user accounts or newly listed items lacking transactional history, the Collaborative Filtering algorithm's accuracy drops, requiring reliance on text-based Content-Based filtering.
- **Budget Tracking Projections:** The forecasting engine uses a simple budget tracking model that requires consistent spending histories. Impulsive, large, or irregular transactions (e.g., tuition fees, large asset purchases) will cause forecasting errors.

5.3 Recommendations & Product Roadmap

To transition **Hmall** from an academic prototype to a commercial startup platform, the research team proposes a three-phase deployment plan:



5.3.1. Phase 1: Technical and Security Optimizations (June 2026 - September 2026)

- **Redis Caching:** Integrate a Redis cache in front of Supabase to store frequently accessed product lists and user profiles, reducing database lookup overhead and lowering API response latencies below 50ms.
- **FCM Push Notifications:** Integrate Firebase Cloud Messaging (FCM) to send real-time alerts for budget warnings, task completions, and order tracking updates.
- **Database Indexing:** Build database indexes on lookup fields (user_id, product_id, created_at) within PostgreSQL to ensure fast search response times as transactional records grow.

5.3.2. Phase 2: Live Integrations & AI Upgrades (September 2026 - January 2027)

- **Payment Gateway Connections:** Connect to official payment gateway APIs (VNPay, MoMo, ZaloPay) to support real cash deposits and checkouts.
- **Real-time Online AI Learning:** Shift from periodic batch training to online recommender updates, refining similarity scores immediately upon user interactions.

- **Advanced Spend Forecasting:** Replace simple budget tracking with time-series forecasting models (such as ARIMA or LSTM networks) to isolate irregular spending spikes and raise budget projection confidence scores above 95%.

5.3.3. Phase 3: Commercialization and Market Rollout (January 2027 - June 2027)

- **Merchant SaaS Subscription:** Build a premium dashboard (Vendor Dashboard Premium) that offers vendors consumer demographic insights and AI demand forecasting under a SaaS subscription model.
- **Gamification Systems:** Introduce interactive mini-games to help users earn reward coins, raising Daily Active Users (DAU) and platform retention.
- **Targeted Student Marketing:** Deploy marketing campaigns at major universities to reach Gen Z consumers and student shoppers who benefit most from automated allowance tracking.

REFERENCES

- [1] Bressan, G., & Siqueira, S. W. (2022). Collaborative filtering recommender systems: Analysis of similarity measures. *Journal of Information and Data Management*, 13(2), 154-169
- [2] Cloudinary Inc. (2026). Cloudinary image and video management APIs. *Cloudinary Documentation*. <https://cloudinary.com/documentation>
- [3] Expo. (2026). Expo SDK and tools documentation. <https://docs.expo.dev/>
- [4] Express. (2026). Express - Fast, unopinionated, minimalist web framework for Node.js. *Express Documentation*. <https://expressjs.com/>
- [5] Grover, P. (2021). Linear regression analysis: Theory and practice. *O'Reilly Media*.
- [6] Han, J., Kamber, M., & Jian, P. (2012). Data mining: Concepts and techniques (3rd ed.). *Morgan Kaufmann*.
- [7] JWT. (2026). JSON Web Token introduction and specifications. *JWT.io*. <https://jwt.io/introduction>
- [8] Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. *Cambridge University Press*.
- [9] Express. (2026). Express - A progressive Node.js framework for building efficient, reliable and scalable server-side applications. *Express Documentation*. <https://docs.nestjs.com/>
- [10] Node.js Foundation. (2026). Node.js API documentation and runtime specifications. <https://nodejs.org/docs/latest/api/>
- [11] PostgreSQL Global Development Group. (2026). PostgreSQL 16.0 documentation: Row-level security policies. <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [12] Supabase JS. (2026). Supabase JS ORM: Next-generation Node.js and TypeScript ORM. *Supabase JS Documentation*. <https://www.prisma.io/docs>
- [13] Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. *Proceedings of the 1994 ACM Conference on Computer*

Supported Cooperative Work, 175-186. <https://doi.org/10.1145/192844.192905>

- [14] *Ries, E. (2011). The lean startup: How today's entrepreneurs use continuous innovation to create radically successful businesses. Crown Business.*
- [15] *Supabase. (2026). Supabase: The open source Firebase alternative. Supabase Documentation. <https://supabase.com/docs>*
- [16] *TypeScript. (2026). TypeScript language specification and documentation. Microsoft Corporation. <https://www.typescriptlang.org/docs/>*
- [17] *WordTokenizer & natural. (2026). Natural: General natural language facilities for Node.js. npm Registry. <https://www.npmjs.com/package/natural>*